# Cloud Computing

# Cloud Storage

Prof. Emiliano Casalicchio
Department of Computer Science
Sapienza University of Rome

# Data storage in the Cloud

- There are various form of data storage in cloud systems
  - Distributed file systems
    - Google File System (GFS), Hadoop Distributed File System (HDFS)
    - GFS and HDFS rely on concept early developed in Network File Systems and Parallel File Systems
  - SQL databases
  - NoSQL database (or NoSQL datastore or simply datastore)
    - Key-value storage systems
      - E.g. BigTable, Dynamo
- Common goals
  - Massive scaling on demand
  - High availability
  - Simplified application development and deployment

# Agenda

- Basic concepts
  - Atomicity
  - Consensus protocols
  - Storage models (Cell and journal storage)
- Overview on
  - Google and Hadoop file systems
  - S3
- Deep analysis in the next FC

*Cloud Computing Theory and Practice*, D.C.Marinescu

- Chapter 2 (focus on 2.10, 2.11)
- Chapter 8

+ readings on Google FS, HDFS, BigTable, Dynamo

# Basic concepts

Atomicity

Storage models

# Atomic Transaction

Section 2.10 Marinescu

- a multi-step operation (transaction) that should complete without any interruption (atomic)
- Atomicity requires
  - HW support, i.e. non-interruptable ISA operations
    - **Test-and-set** writes to a memory location and returns the old content of that memory cell as non-interruptible operations
    - **Compare-and-swap** compares the contents of a memory location to a given value and, only if the two values are the same, modifies the contents of that memory location to a given new value
  - Mechanisms to access shared resources and to create critical sections, e.g.
    - Locks, semaphore, monitors

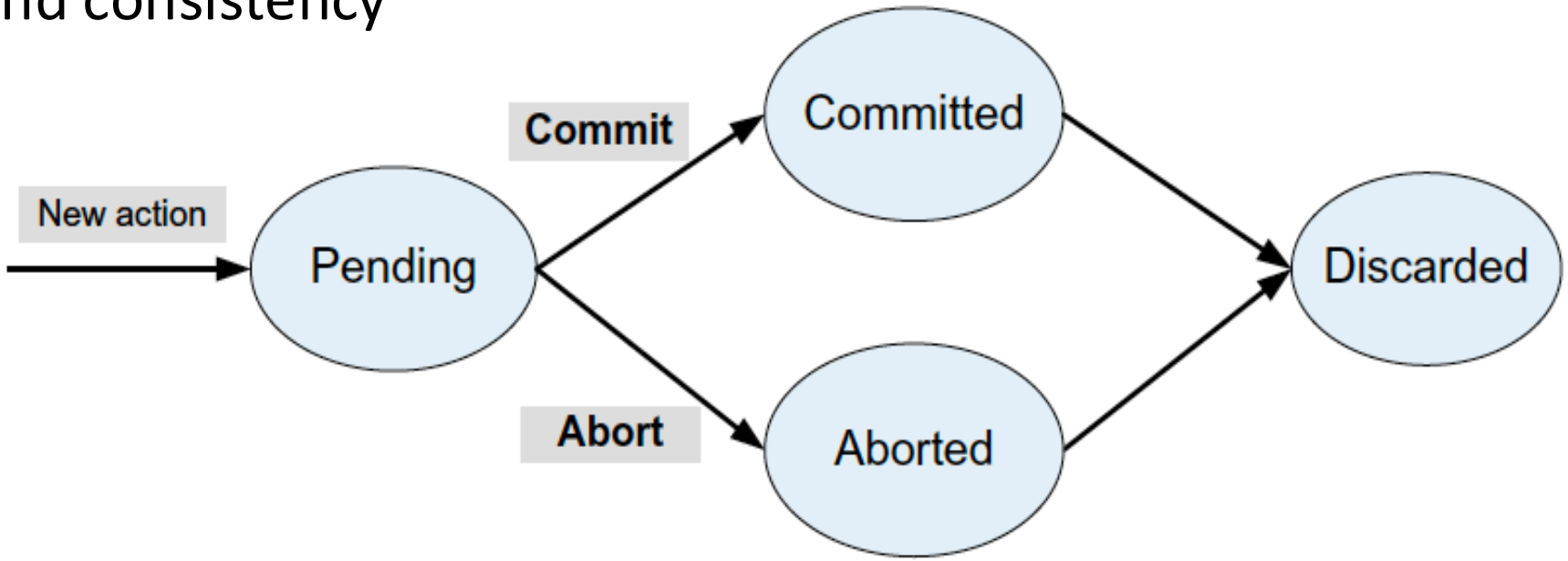
# Atomicity

- All-or-nothing
- Before-or-after

# All-or-nothing atomicity

Chapter 2.10, Marinescu

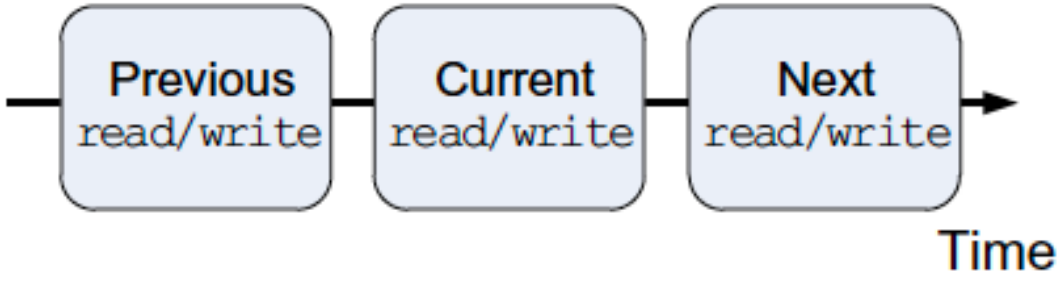
- Two phases
  - **pre-commit phase**; preparatory actions, can be undone
    - allocation of a resource, fetching a page from secondary storage, allocation of memory on the stack
  - transition: commit-point
  - **post-commit phase/commit step**; Irreversible actions
    - alteration of the only copy of an object
- Maintaining an history of changes and a log of actions allows to deal with system failures and consistency



Chapter 2.10 & 8, Marinescu

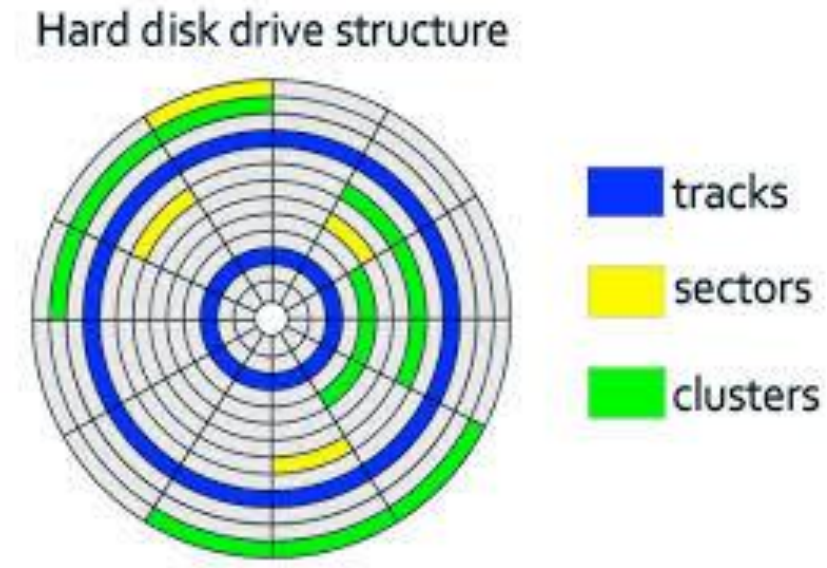
# Before-or-after atomicity

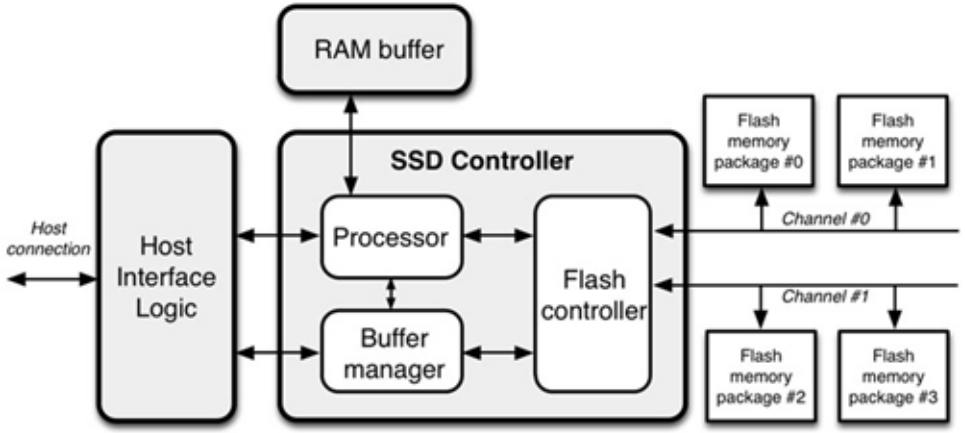
- The result of every read or write is the same as if that read or write occurred either completely before or completely after any other read or write
- E.g. if we have a sequence of read and write operation in a transaction, the concurrent execution of this operation should lead to the same result as if we execute the serialized version of the transaction



# Storage models & desired properties

- Physical storage
  - A local disk, a removable USB disk, a disk accessible via a network
  - Solid state or magnetic
- A *storage model* describes the layout of a data structure in physical storage
  - cell storage
  - journal storage
- Desired properties
  - Read/write coherence
  - Before-or-after atomicity
  - All-or-nothing atomicity



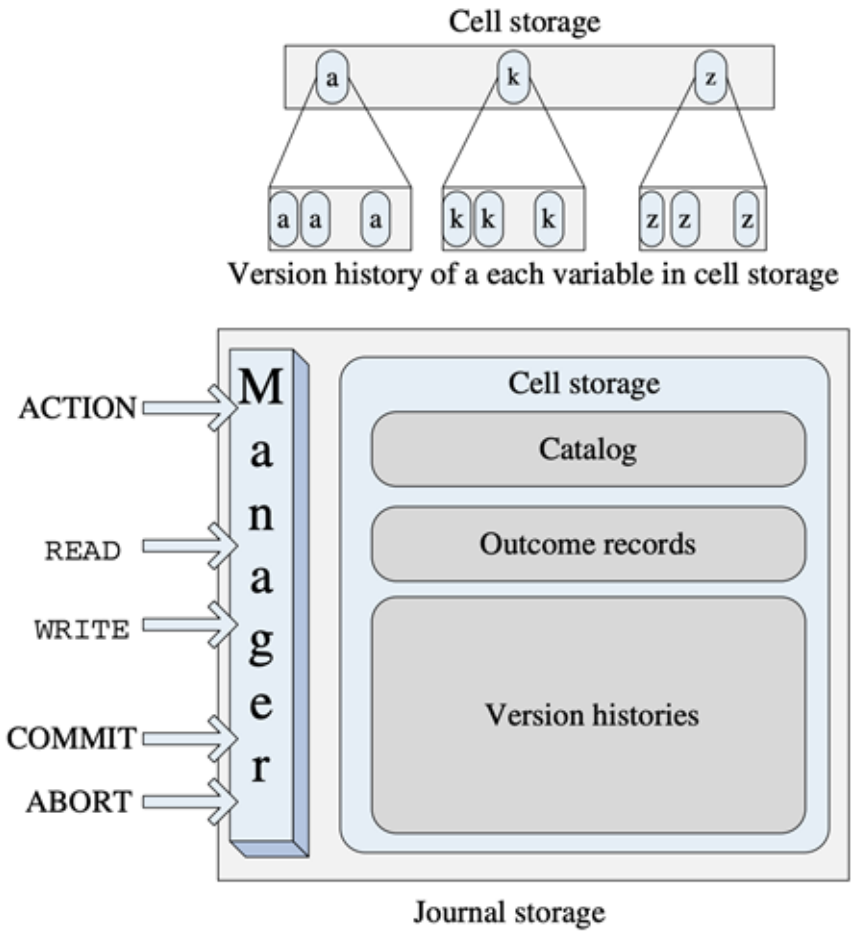



# Cell storage model

- Assumption on the storage
  - cells of the same size
  - each object fits exactly in one cell
  - Read/write unit is sector or block
- Reflects the physical organization of common storage media
  - primary memory - organized as an array of memory cells
  - secondary storage device (e.g., a disk) - organized in sectors or blocks
- Guarantee **read/write coherence**
  - The result of a read of a memory cell M should be the same as the most recent write to that cell

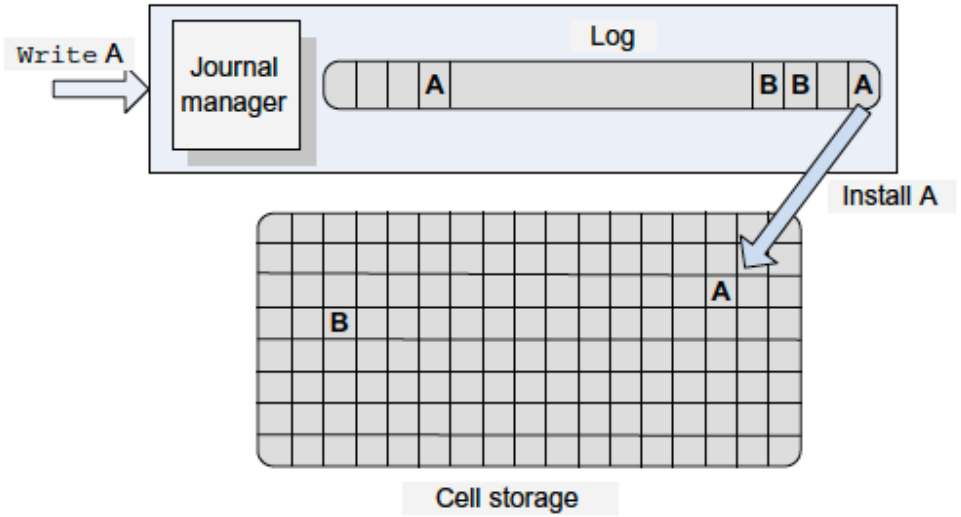
# Journal Storage model

- A model for storing objects like records consisting of multiple fields
- A manager + cell storage
  - The entire history of a variable (not only the current value) is maintained in the cell storage
  - No direct access to the cell storage; the user can request the journal manager to
    - start a new action;
    - read the value of a cell;
    - write the value of a cell;
    - commit an action;
    - abort an action.
  - The journal manager translates user requests to commands sent to the cell storage:
    - read a cell;
    - write a cell;
    - allocate a cell; or
    - deallocate a cell.



# Journal storage model (cont'd)

- The Log contain the history of all variable in the cell store
  - Each update of a data item is a record (containing all the necessary information) appended to the log
  - the log is authoritative
  - allows to reconstruct the cell store
- An all-or-nothing action
  - first records the action in a log in journal storage
  - then installs the change in the cell storage by overwriting the previous version of a data item
- The log is always kept on nonvolatile storage
- The considerably larger cell storage resides typically on nonvolatile memory, but can be held in memory for real-time access or using a write-through cache



# Atomicity and storage models

- Cell storage
  - **does not** guarantee all-or-nothing atomicity
    - Once the content of a cell is changed by an action, there is no way to abort the action and restore the original content of the cell
  - Guarantee
    - read/write coherence
    - before-or-after atomicity

- Journal storage
  - guarantee all-or-nothing atomicity
    - When an action is performed first is modified the version history (log); then is modified the cell store (commit)
    - If one of the two operations fail the action is discarded (abort)
    - All-or-nothing atomicity implies read/write coherence, and before-or-after atomicity
  - guarantee RW coherence and B-o-A atomicity

# Eventual consistency - Database

- Cloud applications
  - application ←→ database ←→ filesystem
  - low latency, scalability, high availability and consistency
- A *database management system (DBMS) should*
  - enforce data integrity
  - manage data access and concurrency control
  - support recovery after a failure
- Relational DBMS
  - guarantees atomicity, consistency, isolation, durability (ACID)
  - Usually not scalable (in speed and size)
- *NoSQL* database models
  - the structure of the data does not require a relational model
  - the amount of data is very large (big data)
  - **may not guarantee the ACID properties**
  - key-value stores, document store databases, and graph databases

# Transaction processing - NoSQL

- D*ocument stores* and *NoSQL* databases
  - designed to scale well,
  - do not exhibit a single point of failure
  - have built-in support for consensus-based decisions
    - Decisions taken on the majority of the votes
  - support partitioning and replication as basic primitives
- The “soft-state” approach in the design of *NoSQL* allows
  - data to be inconsistent
  - Implementation of ACID properties is on the application developer shoulders
- Data will be “eventually consistent” at some future point in time instead of enforcing consistency at the time when a transaction is “committed.”
- Data partitioning among multiple storage servers and data replication
  - they increase availability, reduce response time, and enhance scalability.

# Consensus protocols

- Consensus is the process of agreeing to one of several alternatives proposed by a number of agents
- In distributed system the agents are a set of processes expected to reach consensus on a single proposed value
- Consensus protocols
  - Guarantee freedom from inconsistencies (safety)
  - Do not guarantee fault-tolerace; they cannot guarantee progress
- Paxos is a family of protocols to reach consensus based on a finite state machine approach
- Cores in supporting eventual consistency in NoSQL data stores

# basic Paxos

L. Lamport. "Paxos made simple." ACM SIGACT News, 32(4):51–8, 2001.
https://lamport.azurewebsites.net/pubs/paxos-simple.pdf

- The goal is to get the set of processes (or agent) to reach consensus on a single proposed value
  - The consensus service consists of a set of n processes
  - Clients send requests to agents and propose a value and wait for a response
- Assumptions about the agents and the network:
  - The agents run on processors and communicate through a network;
  - the agents and the network may experience failures, but **not Byzantine failures**

# Byzantine failures

- Byzantine failures imply no restrictions on what errors can be created
  - a failed node can generate arbitrary data, including data that makes it appear like a functioning node to a subset of other nodes
- A Byzantine failure in a distributed system could be
  - **Omission failure**, such as
    - crash failure
    - failure to receive a request or to send a response.
  - **Commission failure**, such as
    - processing a request incorrectly
    - corrupting the local state
    - sending an incorrect or inconsistent response to a request

# basic Paxos (cont'd)

L. Lamport. "Paxos made simple." ACM SIGACT News, 32(4):51–8, 2001.
https://lamport.azurewebsites.net/pubs/paxos-simple.pdf

- Assumptions about the agents and the network:
  - The agents run on processors and communicate through a network;
  - the agents and the network may experience failures, but **not Byzantine failures**

**The agents:**
(i) operate at arbitrary speeds; may fail by stopping, and may restart.
(ii) Since all agents may fail after a value is chosen and then restart, a solution is impossible unless some information can be remembered by an agent that has failed and restarted.
- Processors have stable storage and agents may rejoin the protocol after a failure;
(iii) can send messages to any other processor.

**The network:**
(i) may lose, reorder, or duplicate messages;
(ii) sends messages are asynchronously that may take arbitrarily long times to reach the destination
(iii) messages are not corrupted

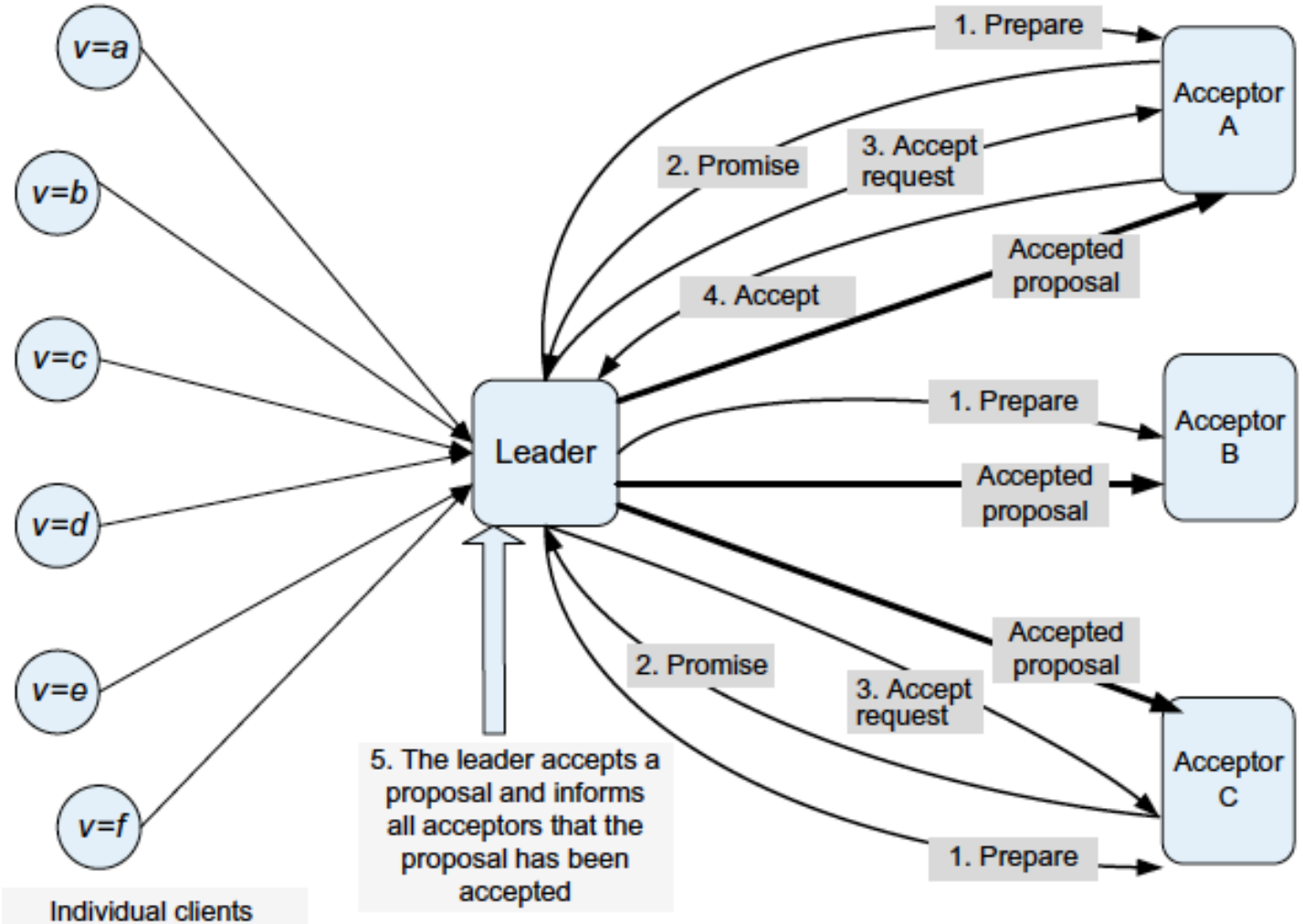
# basic Paxos agents

- **Client**
  - issues a request and waits for a response
- **Proposer**
  - advocates a request from a client
  - convinces the acceptors to agree on the value proposed by a client
  - acts as a coordinator to move the protocol forward in case of conflicts
- **Leader**
  - a distinguished proposer
- **Acceptor**
  - acts as the fault-tolerant “memory” of the protocol
- **Learner**
  - acts as the replication factor of then protocol and takes actions once a request has been agreed upon

# Quorum and Proposal

- A quorum is a subset of all acceptors
- A proposal consists of a pair, a unique proposal number and a proposed value *(pn, v)*
  - multiple proposals may propose the same value *v*
  - a value is chosen **if a simple majority of acceptors** have accepted it
- In a typical deployment of an algorithm, an entity/node plays three roles:
  - **proposer, acceptor, and learner**

Message flow in basic Paxos




1. Prepare – the leader chooses a proposal with proposal number pn=k.
2. Promise – an acceptor promises to accept the proposal only if it has not responded to a proposal with *pn* > *k*. (B does not respond)
3. Accept request – the leader chooses the value *v* of the highest proposal number from all acceptors who have sent a promise and sends it to all of them.
4. Accept – an acceptor accepts a proposal with *pn*=*k* only if it has not promised to accept a proposal with *pn* > *k*. Stores the accepted value in persistent memory.
5. The leader accepts a proposal if the quorum of acceptors send an accept message for the proposal with *pn*=*k*. (C does not accept)

# basic Paxos algorithm: phase I

- (Prepare) Proposal preparation:
  - A proposer (the leader) sends a proposal (pn = k, v) and sends a **prepare message** to a majority of acceptors requesting:
    - a. that a proposal with *pn* < *k* should not be accepted;
    - b. the *pn* < *k* of the highest number proposal already accepted by each acceptor.
- Proposal promise:
  - An acceptor must remember the highest proposal number it has ever accepted as well as the highest proposal number it has ever responded to
  - The acceptor can **accept a proposal** with pn = k if and only if it has not responded to a prepare request with pn > k
  - If it has already replied to a prepare request for a proposal with pn > k, then it should not reply
  - Lost messages are treated as an acceptor that chooses not to respond.

# basic Paxos algorithm: phase II

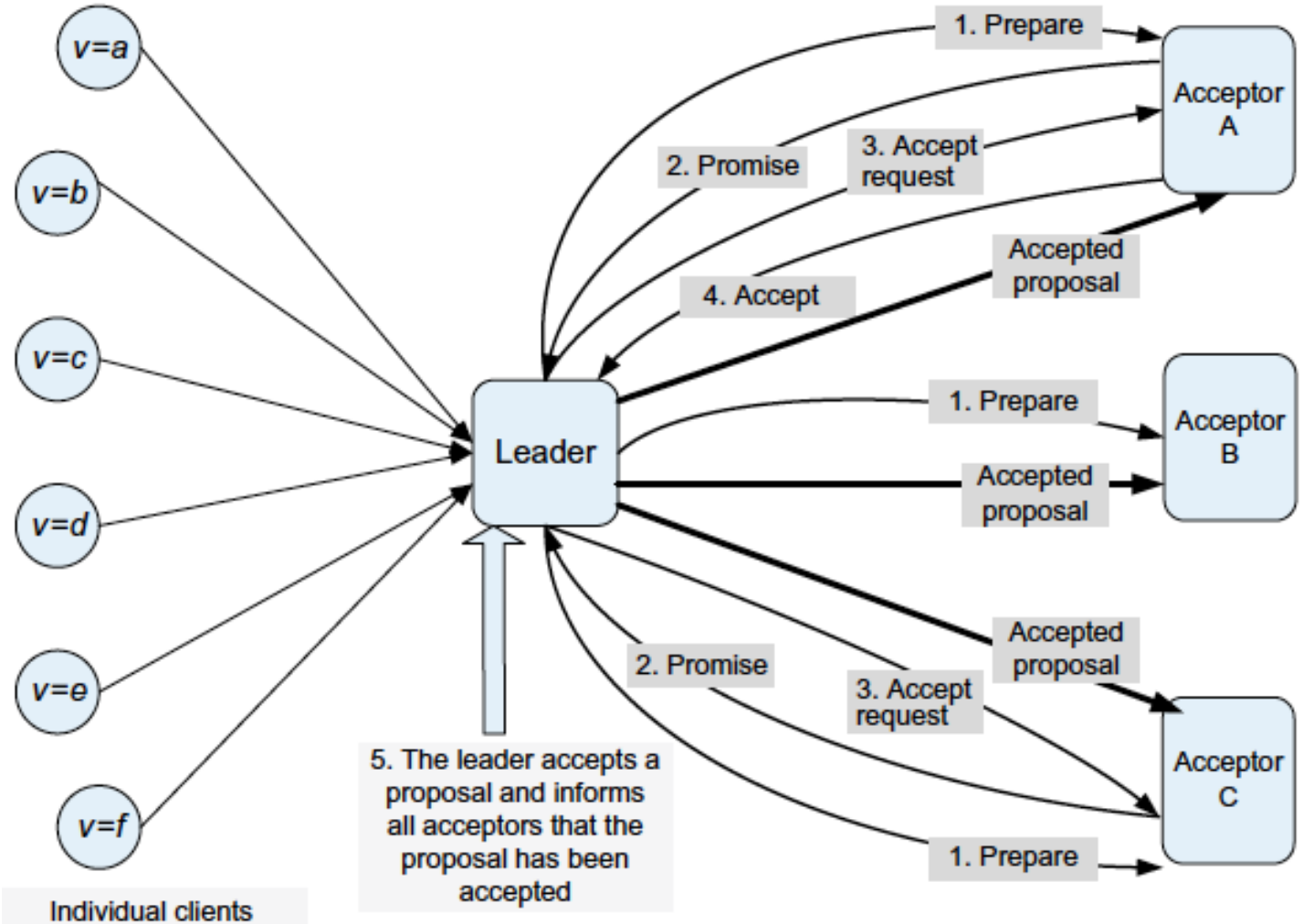
- Accept request:
  - If the majority of acceptors respond, the proposer chooses the value v of the proposal as follows:
    - a. the value v of the highest proposal number selected from all the responses
    - b. an arbitrary value if no proposal was issued by any of the proposers
  - The proposer sends an **accept request** message to a *quorum* of acceptors including (pn = k, v).
- Accept:
  - If an acceptor receives an accept message for a proposal with the proposal number pn = k, it must accept it if and only if it has not already promised to consider proposals with a pn > k
  - If it accepts the proposal, it should register the value v and send an accept message to the proposer and to every learner; if it does not accept the proposal, it should ignore the request.

# Algorithm properties

The following properties of the algorithm are important to show its correctness:

1. a proposal number is unique
2. any two sets of acceptors have at least one acceptor in common
3. the value sent out in Phase II of the algorithm is the value of the highest numbered proposal of all the responses in Phase I

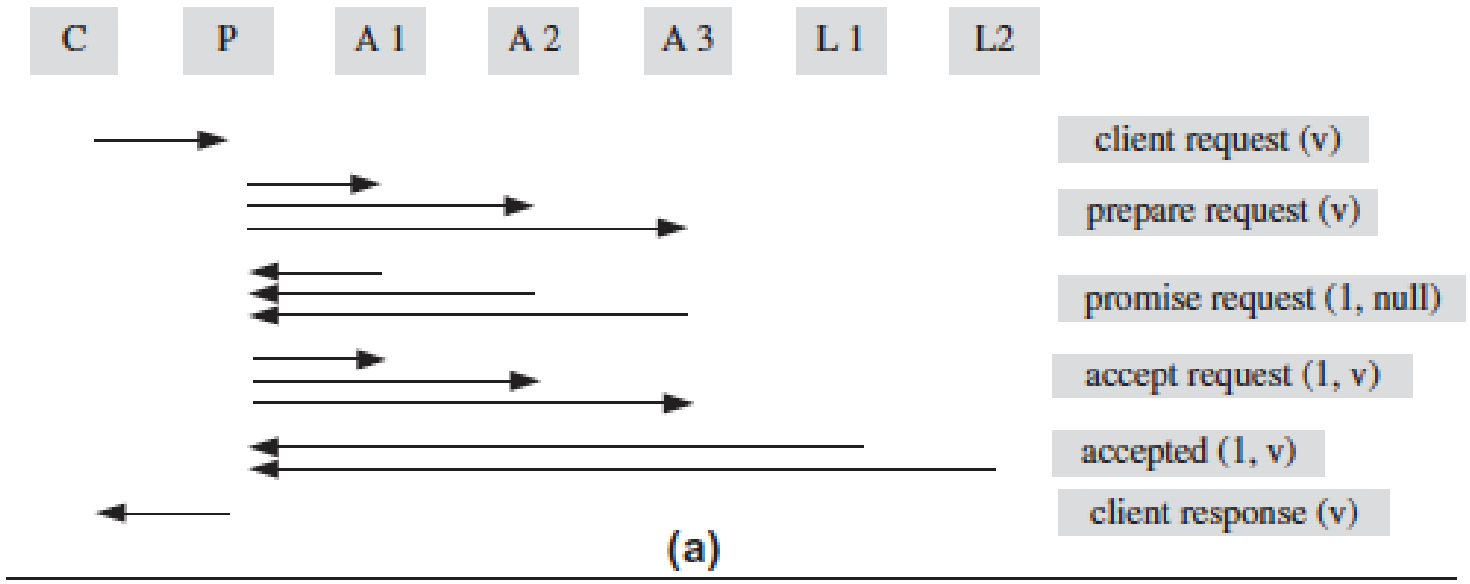
Message flow in basic Paxos



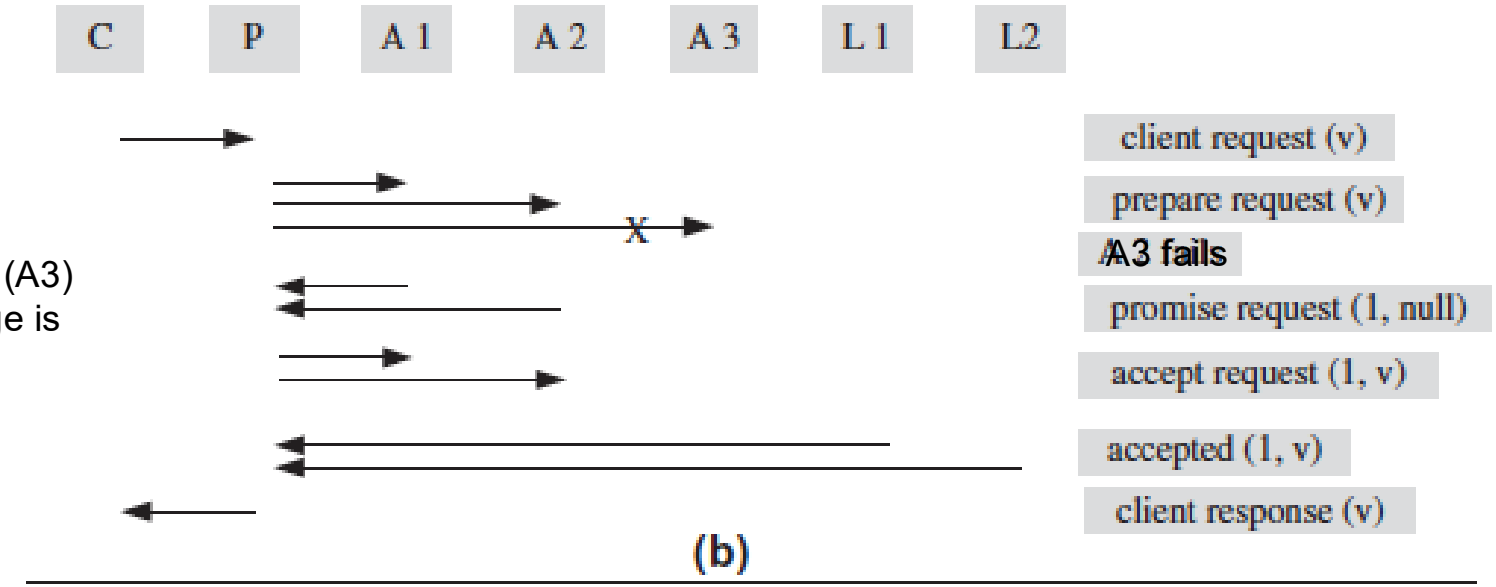

1. Prepare – the leader chooses a proposal with proposal number pn=k.
2. Promise – an acceptor promises to accept the proposal only if it has not responded to a proposal with *pn* > *k*. (B does not respond)
3. Accept request – the leader chooses the value *v* of the highest proposal number from all acceptors who have sent a promise and sends it to all of them.
4. Accept – an acceptor accepts a proposal with *pn*=*k* only if it has not promised to accept a proposal with *pn* > *k*. Stores the accepted value in persistent memory.
5. The leader accepts a proposal if the quorum of acceptors send an accept message for the proposal with *pn*=*k*. (C does not accept)

No failures

promise to accept with pn=1




A node fails (A3) or a message is lost



# High performance, scalable file systems

Google file system

Apache Hadoop

# Google File System (GFS)

- It uses thousands of storage systems built from inexpensive commodity components to provide petabytes of storage
- Design based on
  - high reliability to hardware failures, system software errors, application errors, human errors.
  - a careful analysis of
    - the file characteristics
    - the access models
- Most important features of the ”cloud” access model (check on section 8.5)
  - files range in size from a few GB to hundreds of TB.
  - *append* rather than *random write*
  - *sequential read*
  - *response time* is not the main requirement; data are processed in bulk
  - relaxed consistency model to simplify the system implementation
    - without placing an additional burden on the application developers

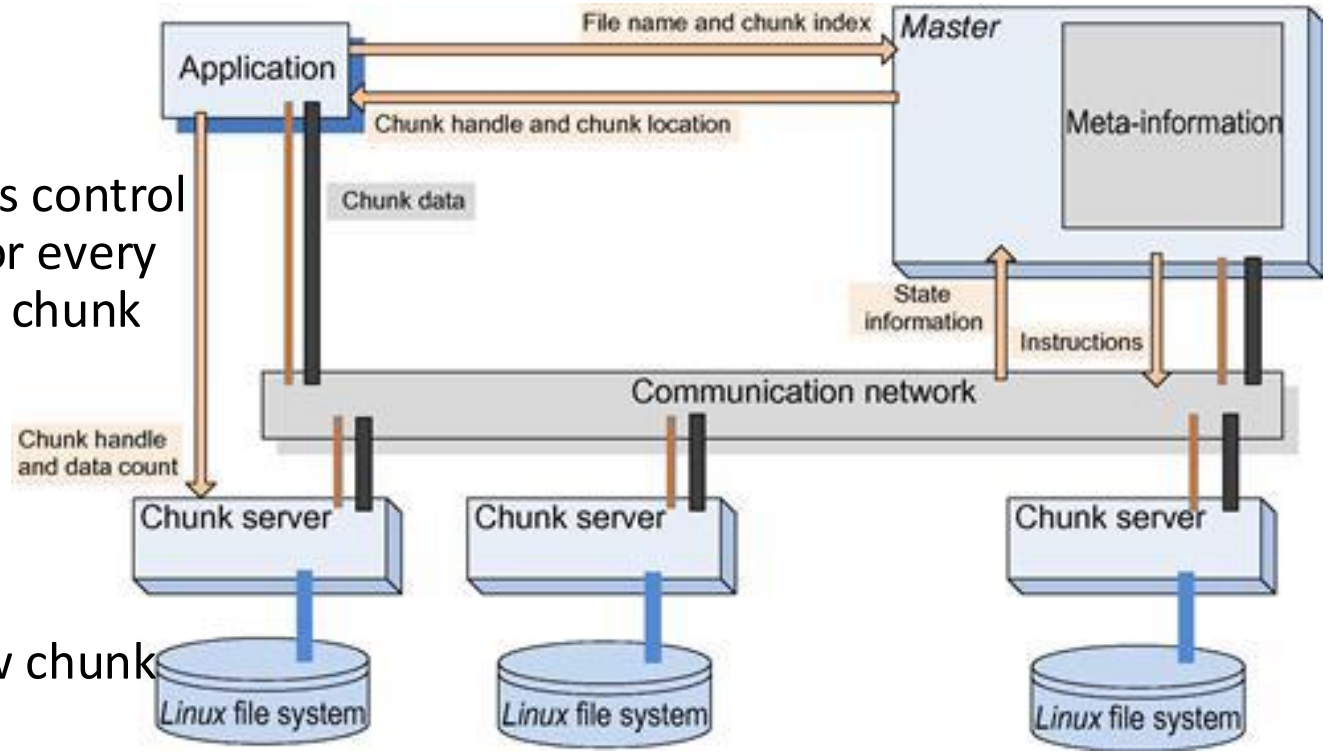
# Files in the GFS

- GFS files are collections of fixed-size segments called chunks
  - chunk size 64MB
    - normal file systems work with blocks of 0.5-2 MB
  - Stored on Linux File System
  - Replicated on multiple sites (3 default – configurable)

- Large chuck size
  - to optimize performance for large files and to reduce the amount of metadata maintained by the system
  - to increase the likelihood that multiple operations will be directed to the same chunk
  - to reduces the number of requests to locate the chunk;
  - to maintain a persistent network connection with the server where the chunk is located
  - to reduce disk fragmentation
    - chunk for a small file and the last chunk of a large file are only partially filled

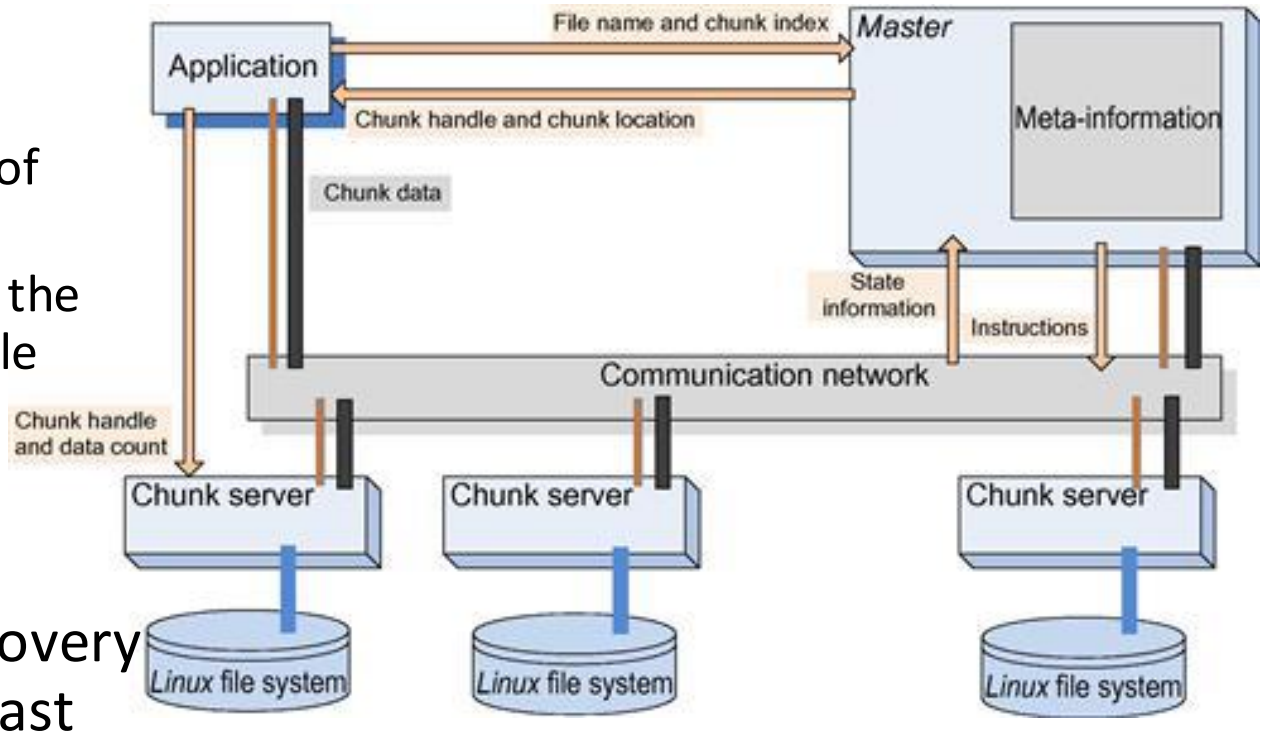
# GFS Cluster Architecture

- A master
  - controls a large number of chunk servers
  - maintains metadata such as filenames, access control information, the location of all the replicas for every chunk of each file, and the state of individual chunk servers
- The locations of the chunks
  - are stored only in the control structure of the *master*'s memory
  - are updated at system startup or when a new chunk server joins the cluster
- This strategy allows the *master* to have up-to-date information about the location of the chunks



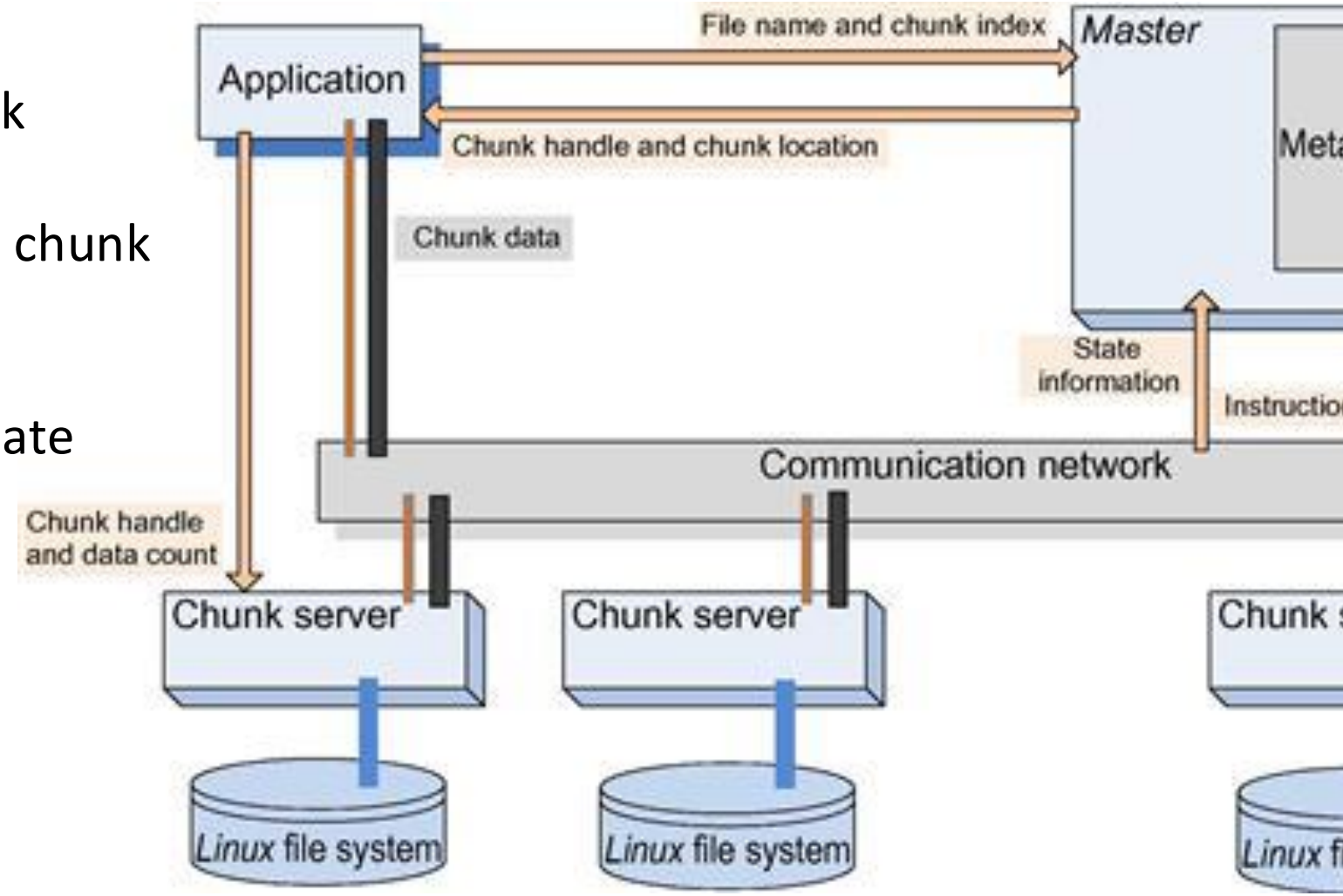
# GFS Cluster Architecture (Cont'd)

- System reliability is a major concern
- To recover in case of failure
  - The operation log maintains a historical record of metadata changes
  - changes are atomic and are not made visible to the clients until they have been recorded on multiple replicas on persistent storage
  - the master replays the operation log
- To minimize the recovery time, the master periodically checkpoints its state and at recovery time replays only the log records after the last checkpoint



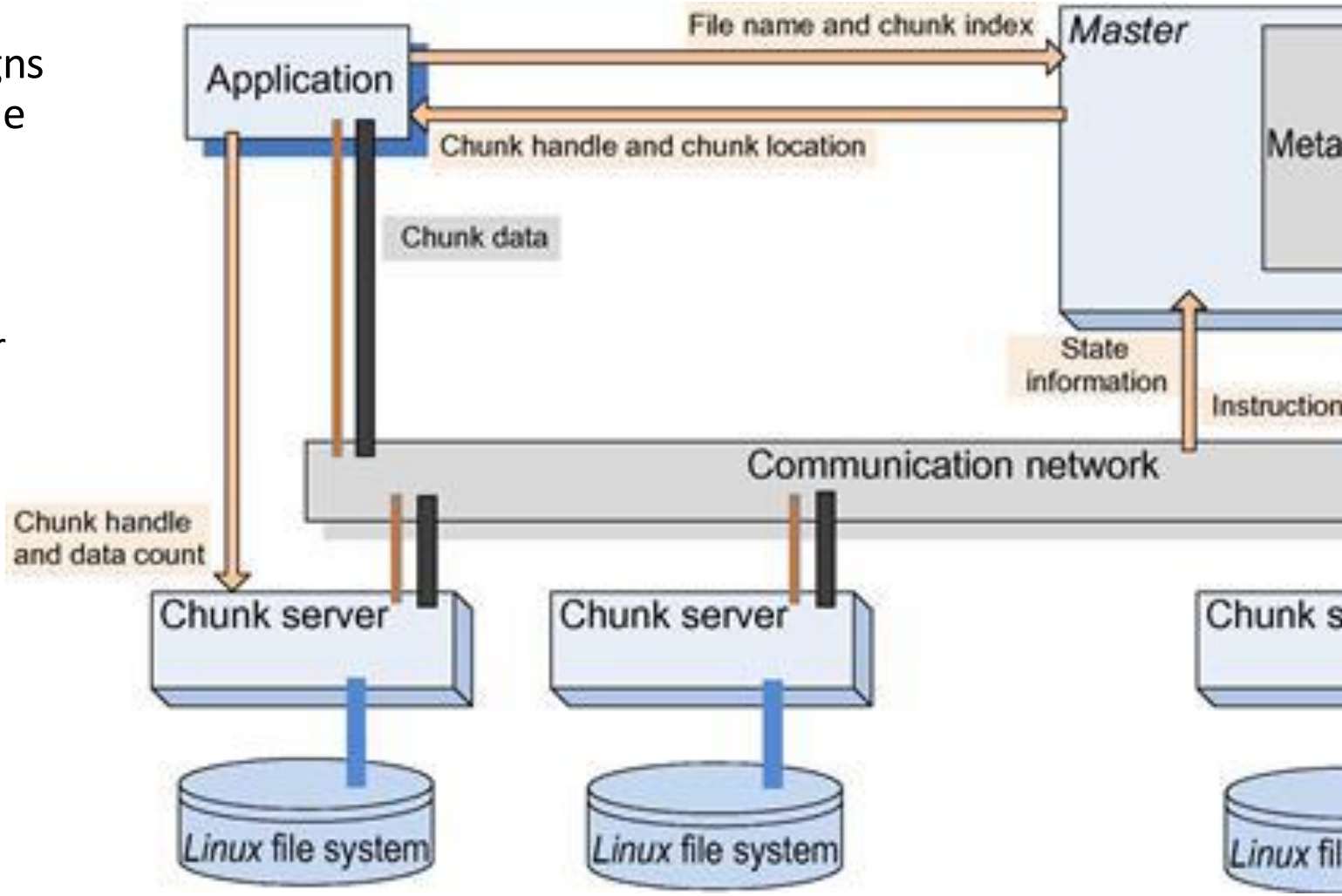
# GFS File Access

- File access
  - Handled by the App and Chunk server
  - The master grants a lease to a chunk server (primary)
  - The primary chunk server is responsible to handle the update
- File creation
  - Handled by the master



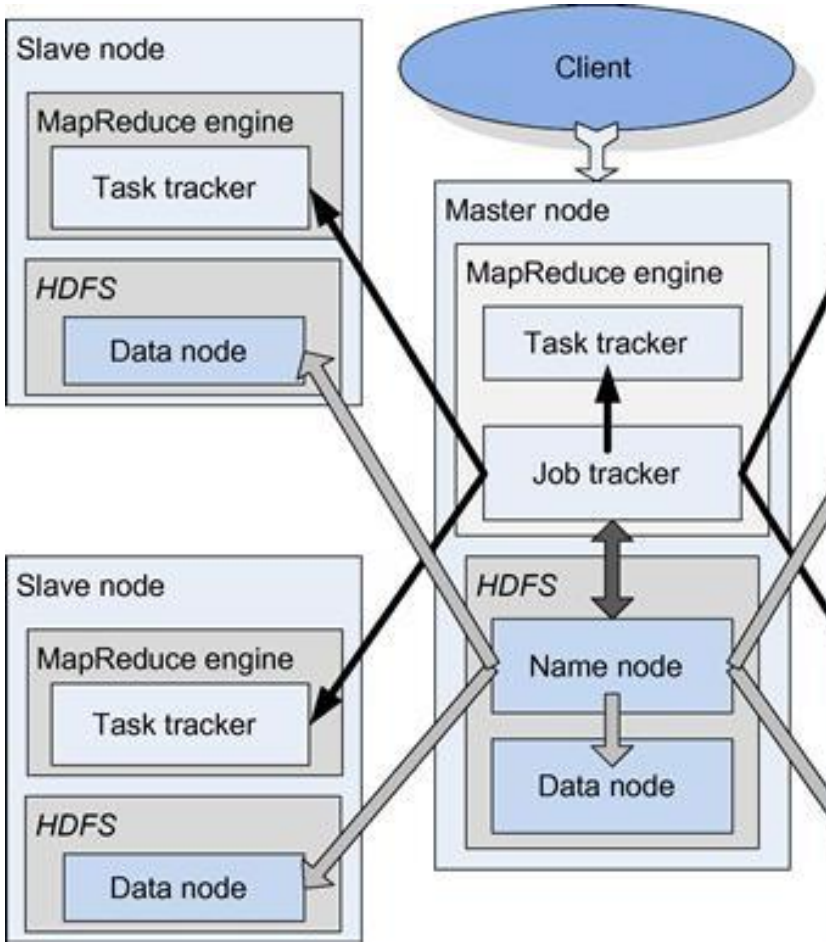
# GFS Write on a File

1. Client contacts the master which assigns a lease to one of the chunk servers (the primary) and reply with ID of primary and secondary chuck servers
2. Client send data to all chunk servers
   1. Chuck server store data in a LRU buffer
   2. Chunk servers send ack to client
3. Client send write req to primary
   1. Primary applies mutation to file
4. The primary send write to secondary
5. Each secondary send ack to primary after mutation are applied
6. The primary ack the client



# Hadoop Distributed File System (HDFS)

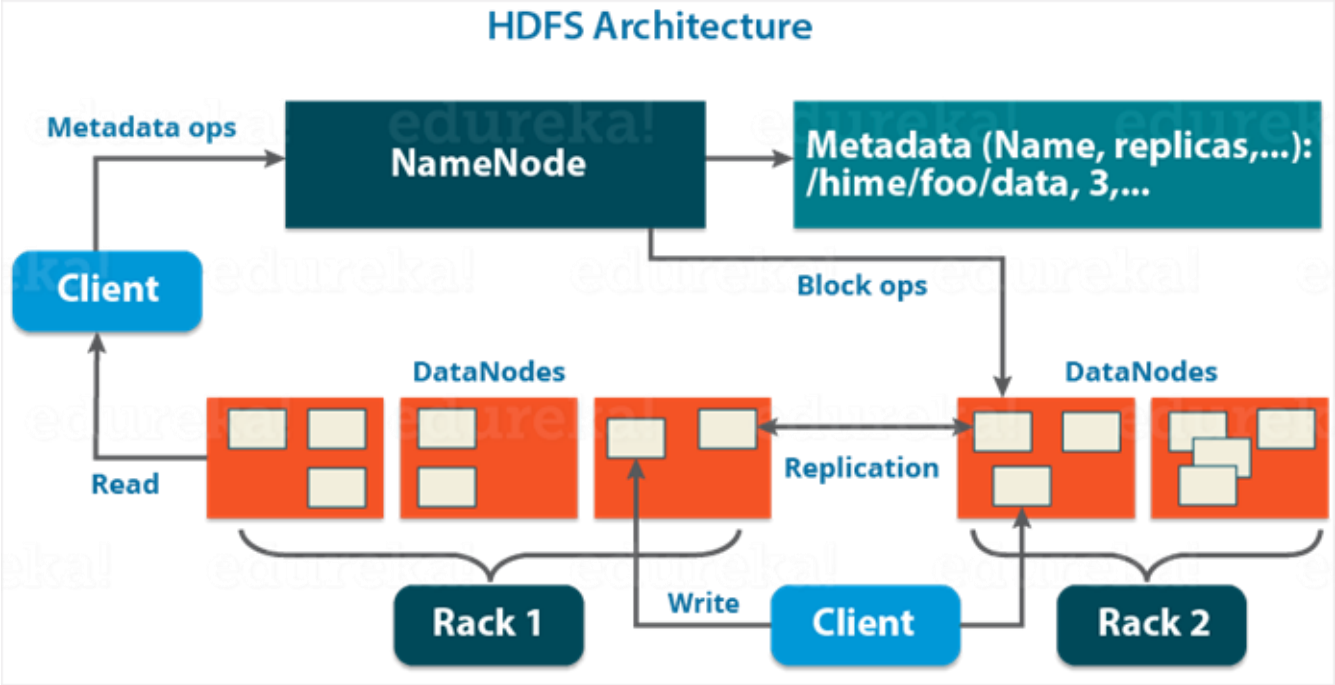
- Apache Hadoop – software system to support distributed applications processing extremely large volumes of data (big data applications)
  - To implement dataflow/pipe&filter SW architectures
  - MapReduce + datastore (HDFS, Amazon S3, CloudStore, …)
- *HDFS is a* distributed fs written in Java
  - it is portable, but it cannot be directly mounted on an existing operating system.
  - not fully POSIX compliant, but it is highly performant
  - 64-128MB block size
- Replicates data on multiple nodes
  - three replicas default
  - large dataset distributed over many nodes



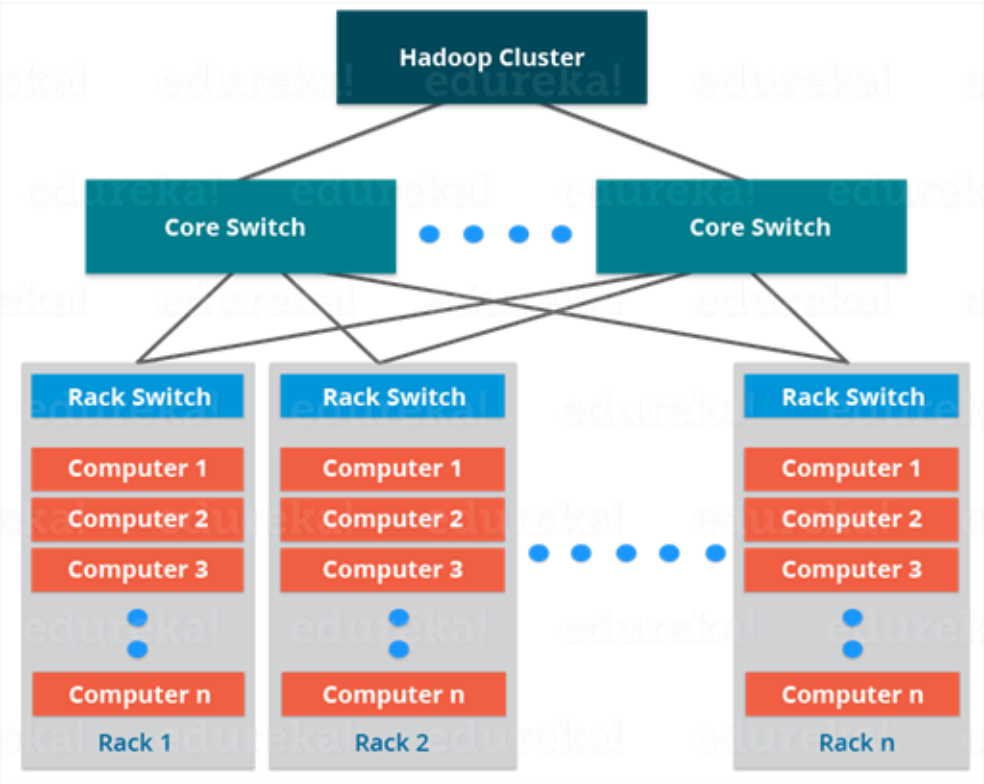
# HDFS Architecture

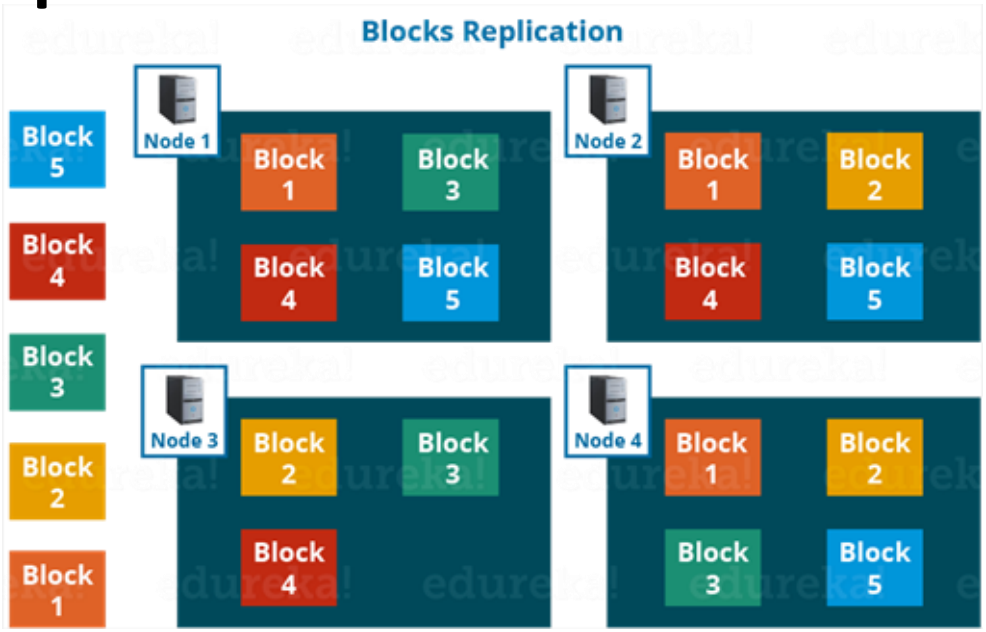
https://www.edureka.co/blog/apache-hadoop-hdfs-architecture/

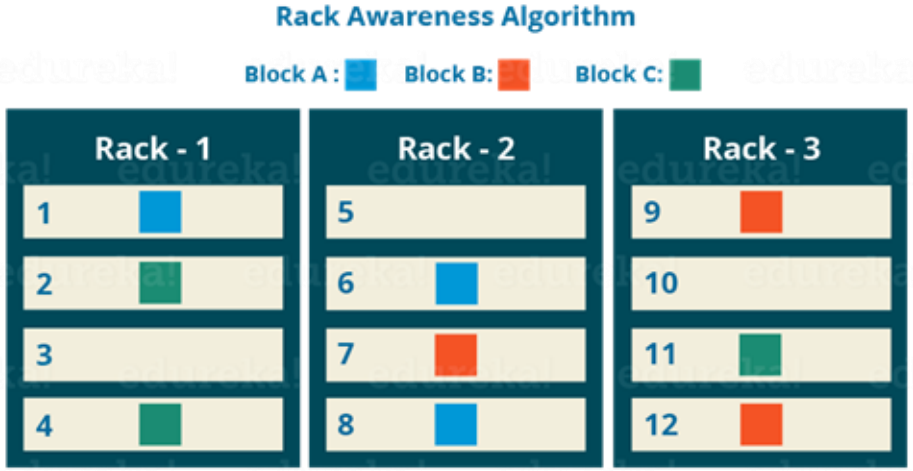
- *Master/Slave Architecture*,
- NameNode (master)
  - manages the File System Namespace
  - store metadata
  - controls access to files
  - record changes in a log
  - check DataNode liveness
  - Secondary Name node for high availability
- DataNode (slave)
  - Low level R/W ops



# HDFS cluster and block replica



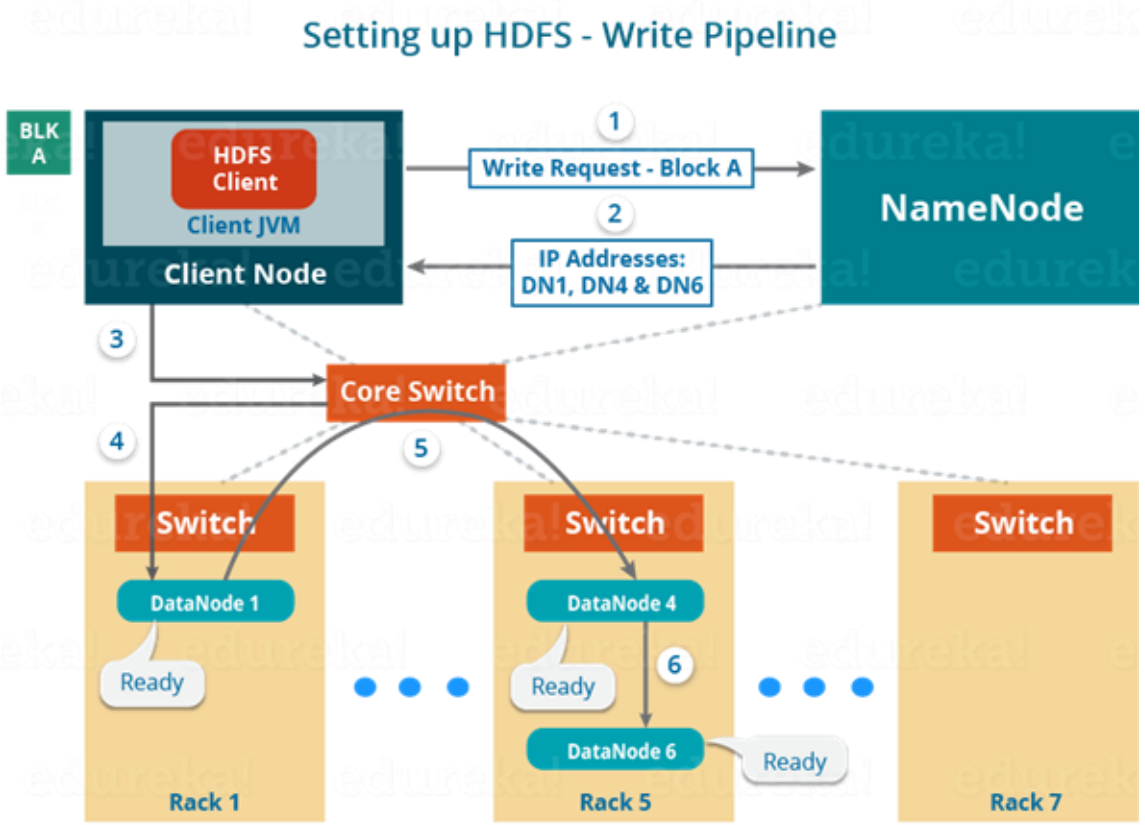



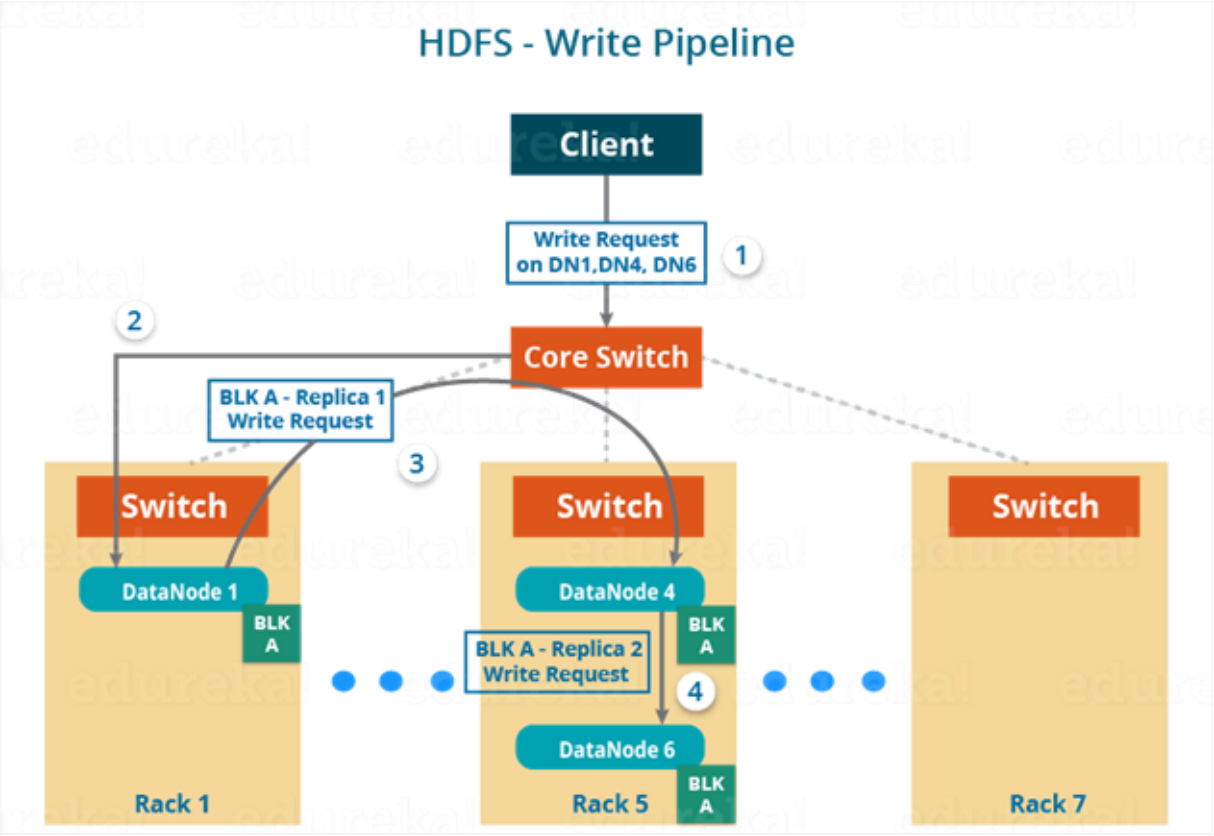

https://www.edureka.co/blog/apache-hadoop-hdfs-architecture/

# HDFS write protocol

- Three main stages
  - Set up of Pipeline
  - Data streaming and replication (Write pipeline stage)
  - Shutdown of Pipeline (Acknowledgement stage)

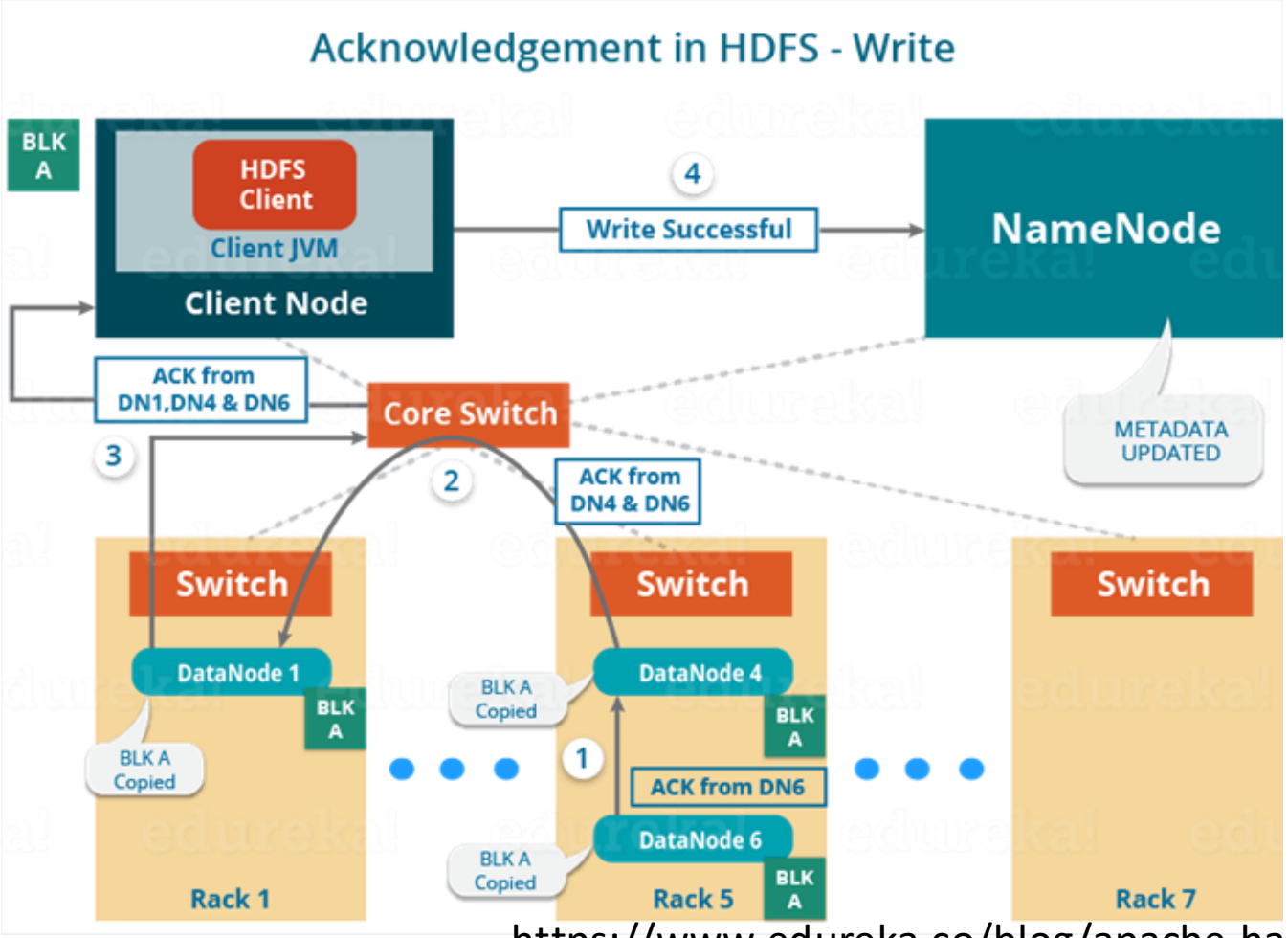
# HDFS write protocol (cont'd)



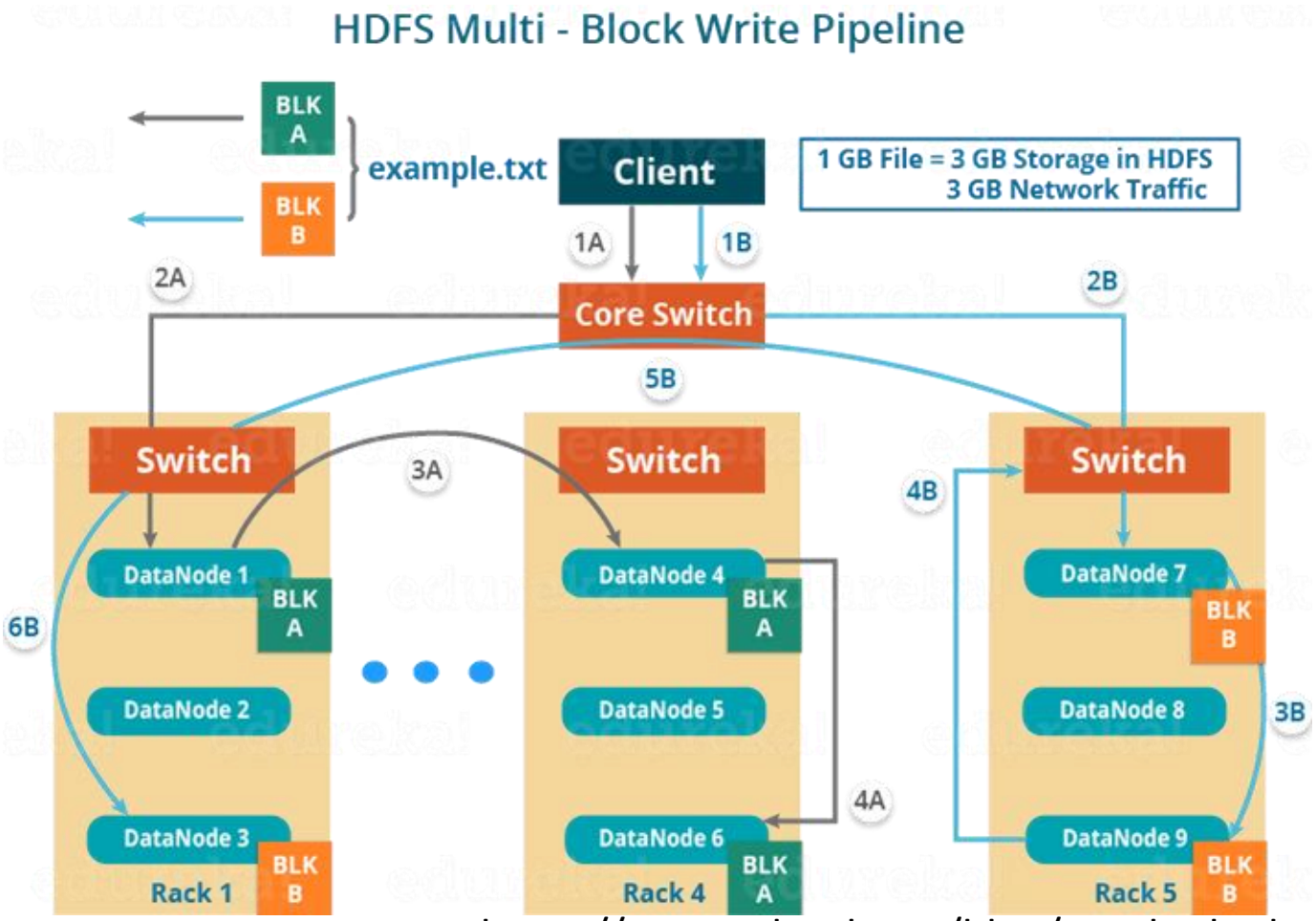

https://www.edureka.co/blog/apache-hadoop-hdfs-architecture/

# HDFS write protocol (cont'd)




https://www.edureka.co/blog/apache-hadoop-hdfs-architecture/

# HDFS write protocol (cont'd)




https://www.edureka.co/blog/apache-hadoop-hdfs-architecture/

# HDFS Read Architecture and Protocol

- Study by yourself
  - https://www.edureka.co/blog/apache-hadoop-hdfs-architecture/

# Object Store

Amazon S3 - Simple Storage Service

# S3 storage model

- Data stored as named “objects” that are grouped in named “buckets.”
- A map backet+key+version
- Buckets
  - names are global;
  - each user may create up to 100 buckets;
  - must be explicitly created (via API) before they can be used
  - Can be listed and deleted (via API)
  - ACL to set read/write permissions
- Object
  - can store any byte string 1 – 5 GB
  - Object key or key name is unique in a bucket
  - URI path name to identify the object
    - https://**DOC-EXAMPLE-BUCKET**.s3.us-west-2.amazonaws.com/**photos/puppy.jpg**
  - stored and retrieved in their entirety or by specific byte ranges with PUT & GET

# RW failure/error handling

- The application must handle RW failures and errors
- The client must retry and RW request until succeed
- MD5 checkum returned in the Etag used to check integrity

# Write failures

- Write retry
  - The service can fail when an HTTP PUT is issued, forcing a retry
- Write error
  - S3 computes the MD5 of every object that is written and returns the MD5 as part of the write acknowledgment in a special field called ETag
  - Clients are supposed to compute the MD5 of objects that they write and compare this MD5 with the returned ETag
  - If the two values do not match, then the object was corrupted during transmission or storage
  - In this case, the object should be re-written

# Read failures

- Read Retry
  - Instead of returning the requested data, S3 can generate some kind of error, forcing a retry
- Read Error
  - When S3 returns a data object, the MD5 of the object is returned in the ETag field
  - If the MD5 does not match the original value stored, a read error has taken place

# S3 data consistency model for objects

- Strong read-after-write consistency for PUT and DELETE requests of objects
  - both writes to new objects as well as PUT requests that overwrite existing objects
- Updates to a single key are atomic
  - Performing a PUT request to an existing key from one thread and performing a GET request on the same key from a second thread concurrently, will return either the old data or the new data, but never partial or corrupt data
  - PUT return successfully after all the replica are updated
- Strong consistency for Read operations on
  - access controls lists (ACLs), Object Tags, and object metadata

# S3 data consistency model for objects (cont'd)

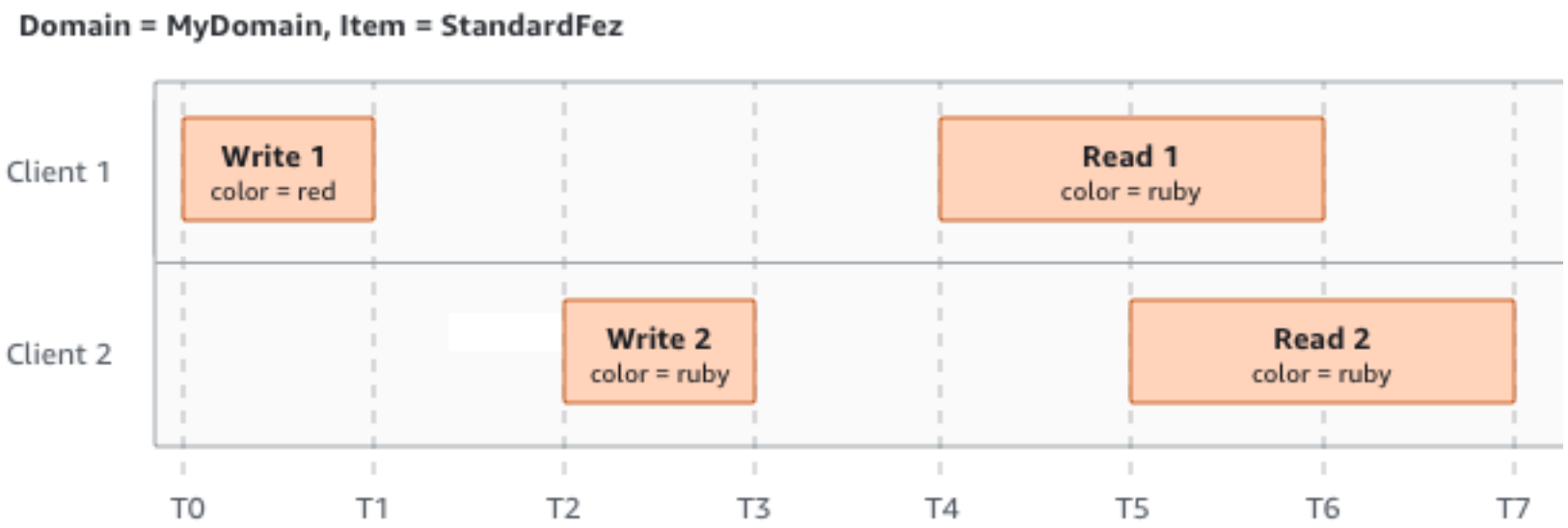
- Amazon S3 does not support object locking for concurrent writers
  - If two PUT requests are simultaneously made to the same key, the request with the latest timestamp wins. If this is an issue, programmers must build an object-locking mechanism into the application.
- Updates are key-based. There is no way to make atomic updates across keys.
  - Not possible to make the update of one key dependent on the update of another key unless this functionality is not designed in the client application.

# S3 data consistency model for buckets

- Bucket configurations have an eventual consistency model
  - After deleting a bucket and immediately listing all buckets, the deleted bucket might still appear in the list.
  - Enable versioning on a bucket for the first time, it might take a short amount of time for the change to be fully propagated
  - Suggested 15 minutes waiting time before performing PUT/DELETE requests on objects

# Examples of concurrent applications behaviour

- Case 1



# Examples of concurrent applications behaviour

- Case 2

# Examples of concurrent applications behaviour

- Case 3

- S3 internally uses last-writer-wins semantics to determine which write takes precedence
- the order in which Amazon S3 receives the requests and the order in which applications receive acknowledgments cannot be predicted
- the best way to determine the final value is to perform a read after both writes have been acknowledged

# NoSQL data store

BigTable

Dynamo

# Database

- Cloud applications
  - application ←→ database ←→ filesystem
  - low latency, scalability, high availability and consistency
- A *database management system (DBMS) should*
  - enforce data integrity
  - manage data access and concurrency control
  - support recovery after a failure
- Relational DBMS
  - guarantees atomicity, consistency, isolation, durability (ACID)
  - Usually not scalable (in speed and size)
- *NoSQL* database models
  - the structure of the data does not require a relational model
  - the amount of data is very large (big data)
  - may not guarantee the ACID properties
  - key-value stores, document store databases, and graph databases

# Transaction processing - OLTP

- Many cloud applications
  - are based on *online transaction processing* (OLTP)
  - operate under tight latency constraints
- Relational databases are not able to handle
  - the massive amount of data
  - the real-time demands
- Major concerns in OLTP systems are
  - (traditional) Response time
    - memcaching, is a solution - a general-purpose distributed memory system that caches objects in main memory (RAM)
  - (cloud applications) Scalability
    - vertical scaling, - always possible
    - horizontal scaling – issue of consistency among multiple distributed copies / portion of the DB

# Transaction processing - NoSQL

- D*ocument stores* and *NoSQL* databases
  - designed to scale well,
  - do not exhibit a single point of failure
  - have built-in support for consensus-based decisions
    - Decisions taken on the majority of the votes
  - support partitioning and replication as basic primitives
- The “soft-state” approach in the design of *NoSQL* allows
  - data to be inconsistent
  - Implementation of ACID properties is on the application developer shoulders
- Data will be “eventually consistent” at some future point in time instead of enforcing consistency at the time when a transaction is “committed.”
- Data partitioning among multiple storage servers and data replication
  - they increase availability, reduce response time, and enhance scalability.

# BigTable

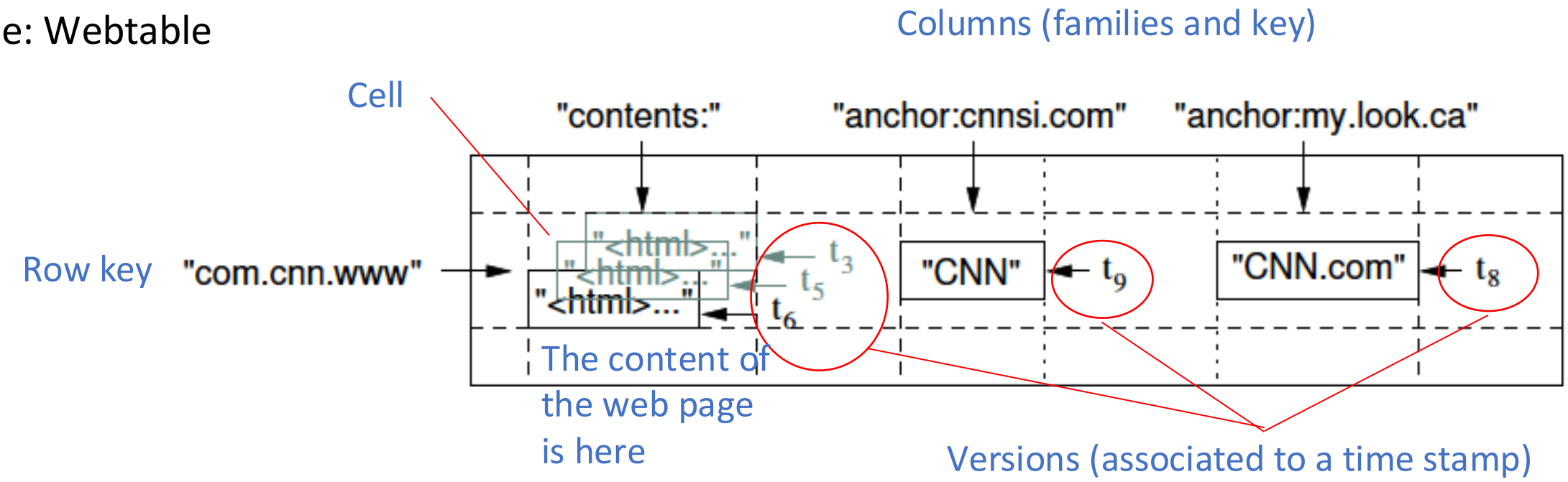
Bigtable: A Distributed Storage System for Structured Data

- BigTable is a distributed storage system developed by Google
  - Achieve wide applicability, scalability, high performance, and high availability
  - Variety of workload
    - from throughput-oriented batch-processing jobs to latency-sensitive serving of data to end users

- BigTable is based on a simple and flexible data model
  - Supports dynamic control over data layout and format
  - Clients can control the locality properties of the data represented in the underlying storage by using appropriate schema
  - Data is indexed using row and column names that can be arbitrary strings
  - Data are treated as uninterpreted strings
  - The client can control serving data from memory or disk

# BigTable data model

- A Bigtable is a sparse, distributed, persistent multidimensional sorted map
- The map is indexed by a row key, column key, and a timestamp
  - (row:string, column:string, time:int64) → string
- Each value in the map is an uninterpreted array of bytes (string)
- Example: Webtable

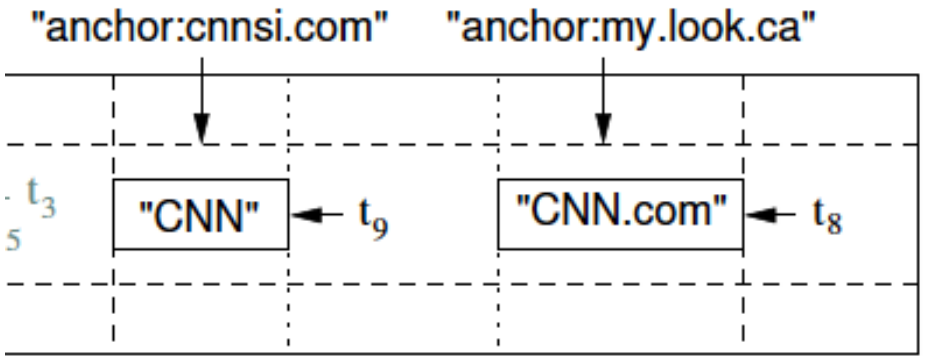


# BigTable Rows

- A row key is an arbitrary string of up to 64 KB (10 - 100 byte is the typical size)
- R/W is atomic regardless the number of columns
- In a table, row are ordered lexicographically by row key
- A row range is dynamically partitioned into **tablets**, the units for load balancing
- Read of short row ranges are efficient and typically require communication with only a small number of machines
  - Clients can exploit this property to improve locality when accessing data
- Webtable Example:
  - The row key is the reverse host name
  - maps.google.com/index → com.google.maps/index

# BigTable Columns

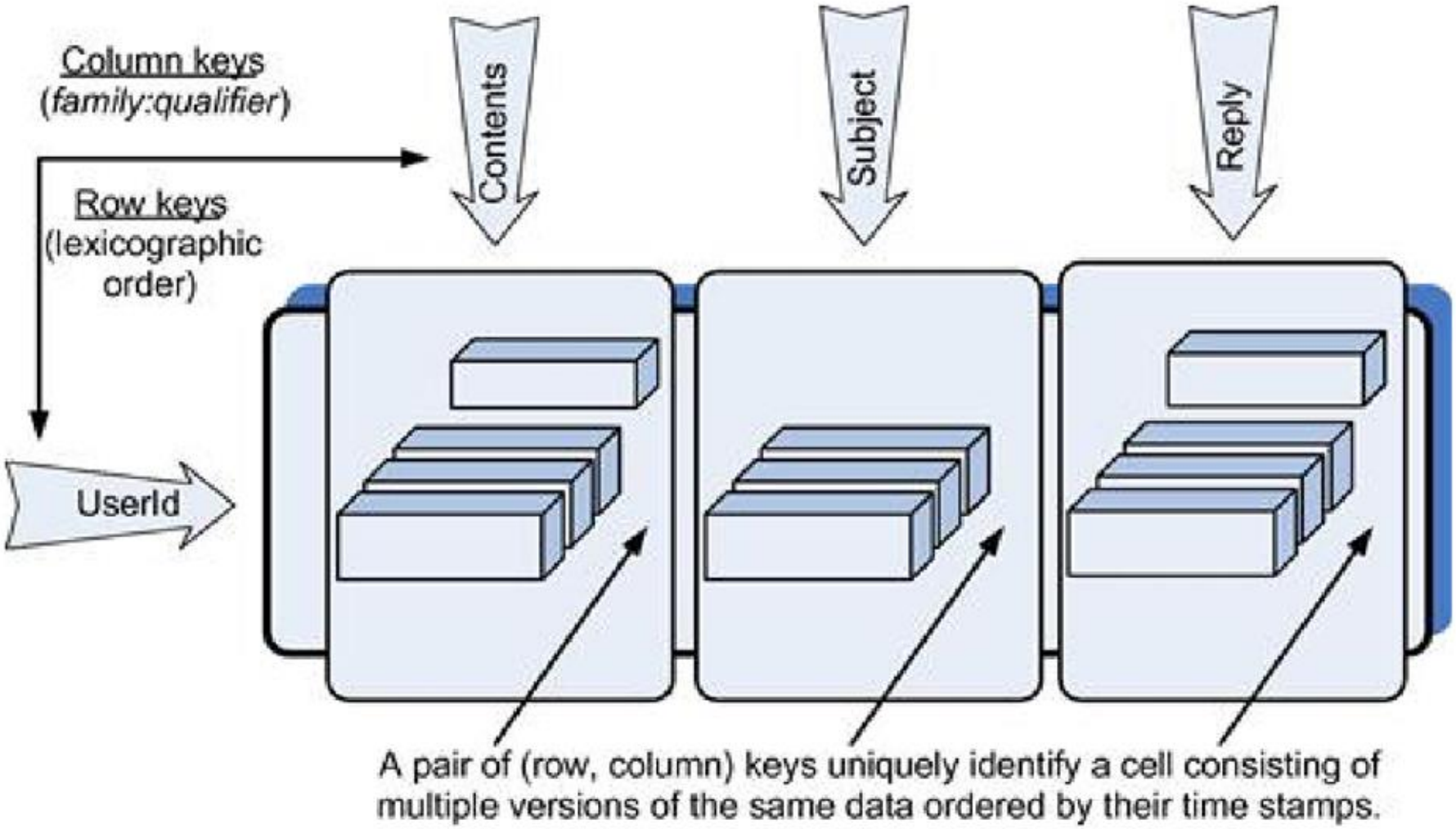
- Column key are grouped into sets called **column families**
  - All the data stored in a column family are of the same type
  - Column family's data can be compressed
  - A family must be created before storing data under any column key
  - The number of family should be small up to hundreds
  - Families should rarely change
  - Number of column in a family unbonded
  - Number of column = number of families * number of keys per family
- A column key is *family:qualifier*
- Example of families
  - Language
  - Anchor
- Example of key
  - *Language:ID*, content of the cell is IT, EN, SW, FR, ….
  - *Anchor:<link_of_referring_site>,* content the link text
- Access control and disk/memory accounting is done at column family level



# BigTable Timestamps

- Each cell can contain multiple versions of the same data
- Versions are indexed by timestamp (64-bit integers)
- Timestamp assignment
  - by BigTable; represent real time in microseconds
  - by the client application
    - Applications that need to avoid collisions must generate unique timestamps themselves
- Different versions of a cell are stored in decreasing timestamp order
  - the most recent versions can be read first
- Two mechanisms to garbage collect cell versions automatically
  - only the last n versions of a cell be kept
  - only new-enough versions be kept (e.g. latest 7 days)
- Example
  - Timestamp of contents column is “the times at which these page versions were actually crawled”
  - Only the 3 most recent versions are kept

# An example of Big Table for the Gmail system



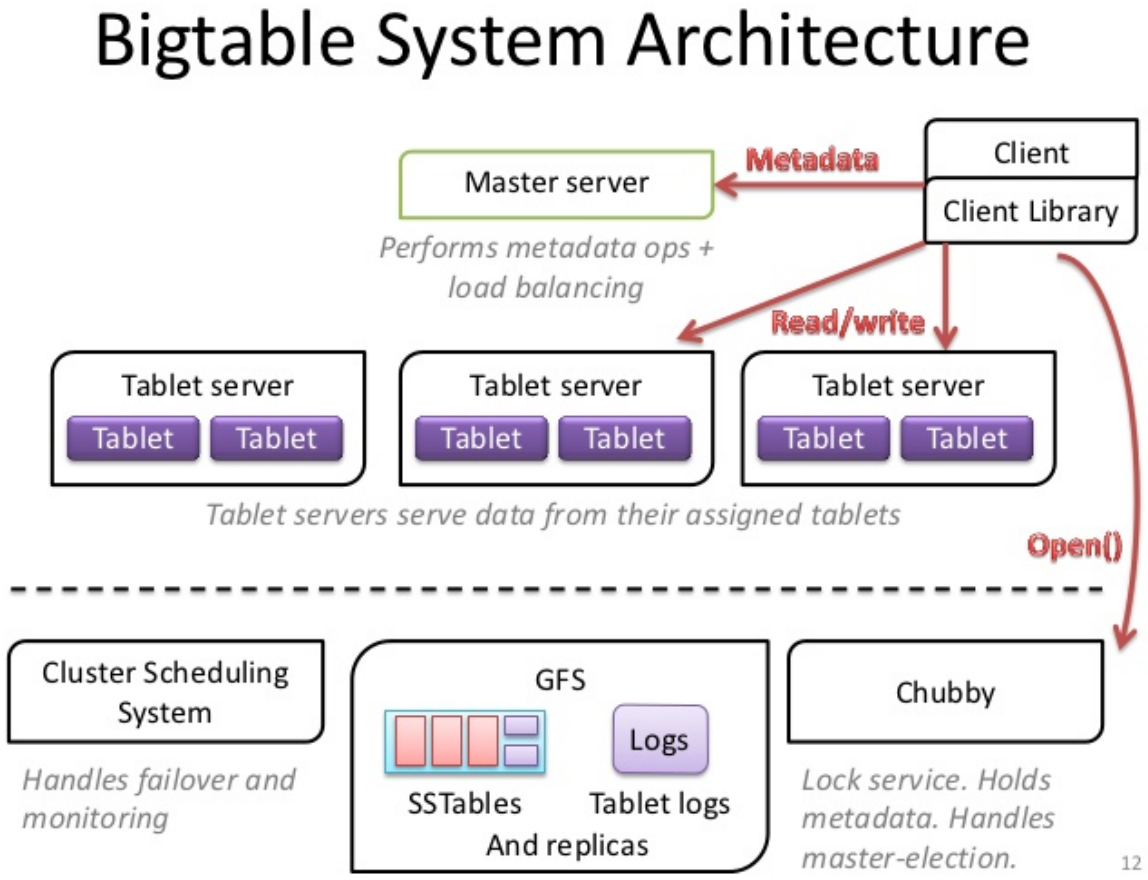

R

# BigTable building blocks

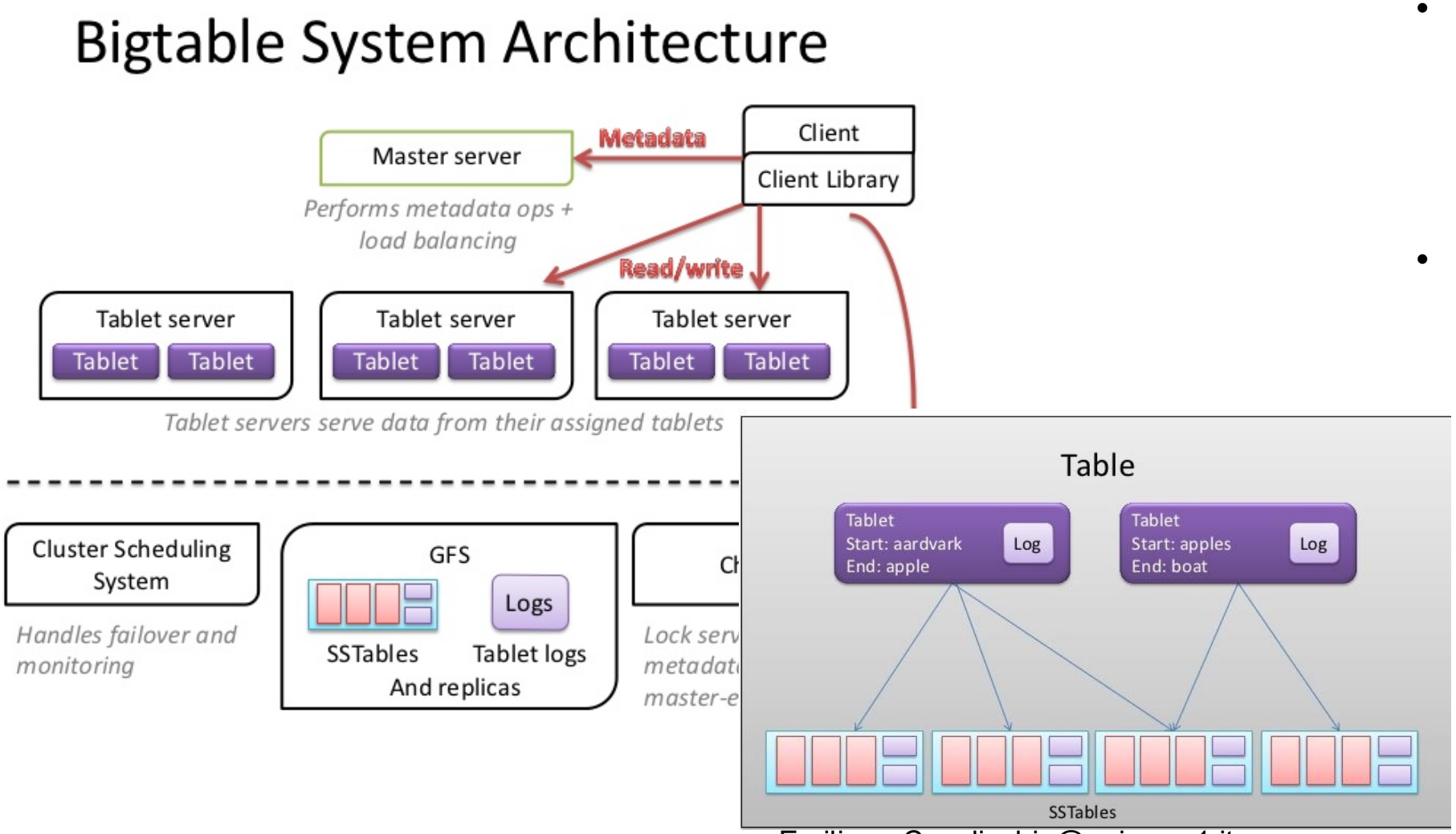
- Uses Google File System
- Google **SSTable** file format
- Chubby, a highly-available and persistent distributed lock service (cf. Chp 8 Marinescu)

# BigTable architecture

- Consist of one **master server** and many **tablet servers**
- A BigTable cluster store of a number of tables; each table consist of a set of tablets
- The **master** is responsible for
  - assigning tablets to tablet servers
  - detecting the addition and expiration of tablet servers
  - balancing tablet-server load, and garbage collection of files in GFS.
  - handling schema changes such as table and column family creations.



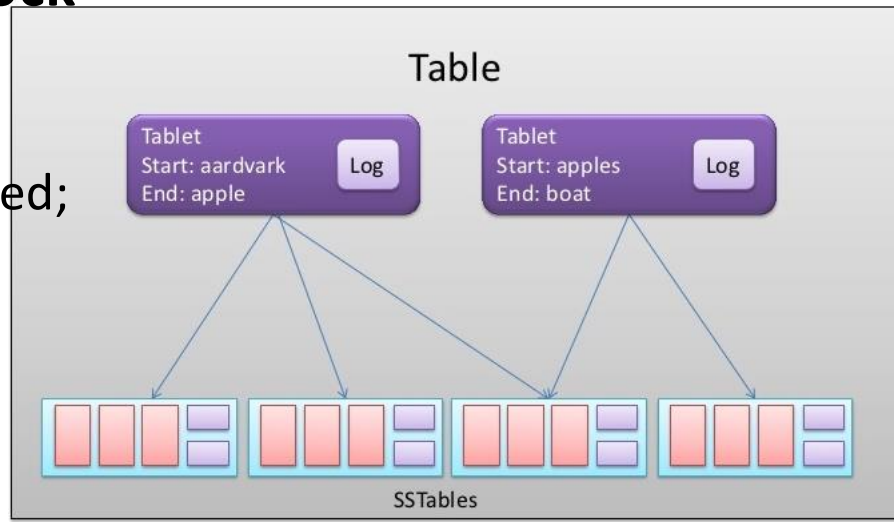
# BigTable architecture (cont'd)




- **Tablet servers** can be dynamically added/removed from a cluster to accomodate the workload
- Each tablet server
  - manages a set of tablets
  - handles read and write requests to the tablets that it has loaded
  - splits tablets that have grown too large (initially for each table there is only one tablet)

# Google SSTable

- Google **SSTable** file format is used internally to store Bigtable data
  - SSTable provides a persistent, ordered immutable map from keys to values
  - Each SSTable contains a **sequence of blocks** and a **block index** to locate blocks
  - To access an SSTable block
    - The index is loaded into memory when the SSTable is opened;
    - The block is found using binary search in memory
    - The block is access with a single seek on disk
  - An SSTable can be also loaded all in memory

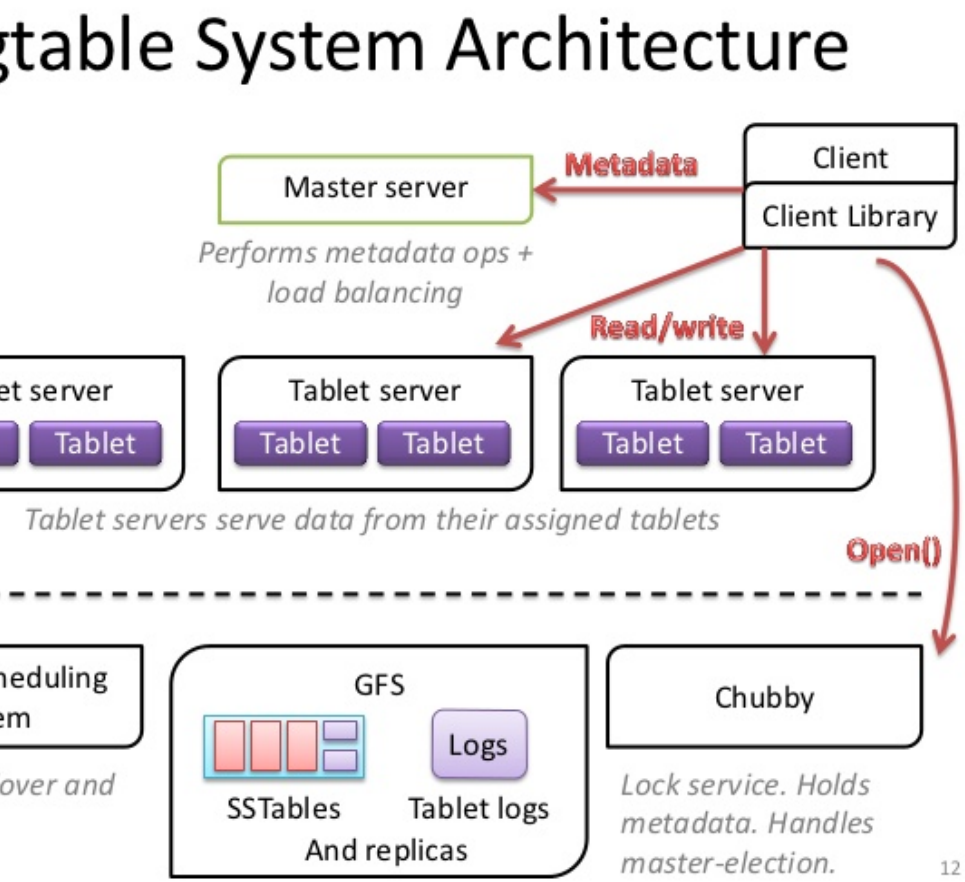


# Chubby (Google inc.)

Read the Chubby papper for details

- A lock service for loosely-coupled distributed system
- The purpose of the lock service is to allow its clients to synchronize their activities and to agree on basic information about their environment.
- Developped to support many services, such as GFS and BigTable. For example:
  - GFS
    - Use Chubby locks to appoint a GFS master server;
    - use Chubby a as a well-known and available location to store a small amount of meta-data;
  - Bigtable uses Chubby in several ways:
    - to elect a master,
    - to allow the master to discover the servers it controls,
    - to permit clients to find the master.
    - as a well-known and available location to store a small amount of meta-data

# Chubby




- Provides a lock mechanisms based on files and directories
- Is used for:
  - To ensure that there is at most one active master at any time
  - To store the bootstrap location of Bigtable data
  - To discover tablet servers and finalize tablet server deaths
  - To store Bigtable schema information (the column family information for each table)
  - To store access control lists

# Amazon's Dynamo

- Highly available and scalable distributed key-value store built for Amazon's platform
  - many services on Amazon's platform that only need primary-key access to a data store
    - best seller lists, shopping carts, customer preferences, session management, sales rank, and product catalog
  - RDB would be inefficiencies and limit scale and availability

- Dynamo is used to manage the state of services that have very high reliability requirements
  - Reliability is one of the most important requirements
  - Outages have significant financial consequences
- Dynamo tradeoffs between availability, consistency, cost-effectiveness and performance.

# Amazon’s Dynamo

- Eventually consistent data store; that is all updates reach all replicas eventually
  - Optimistic replication techniques are used to increase availability in a environment prone to failures
  - In optimistic replication changes are allowed to propagate to replicas in the background
  - a read/write is successful if a minimum number of node R/W can complete successfully the operation
  - optimistic replication can lead to conflicting changes which must be detected and resolved

# Amazon’s Dynamo - requirements

- Query model - simple read and write operations to a data item that is uniquely identified by a key
- ACID (Atomicity, Consistency, Isolation, Durability) properties - ACID DB have poor availability.
  - Dynamo works with weak consistency to improve availability
- Efficiency - dynamo run on commodity HW and should meet latency and throughput requirements
- No specific security requirements - designed to work interlay at Amazon
- Scalability to hundred of storage hosts

# Key design principles - Data Replication

- Synchronous replica coordination leads to poor availability of the data
- Optimistic replication techniques instead
  - allow propagation of changes asynchronously
  - Introduce the problem of inconsistency detect and resolve conflict
- When to resolve conflict
  - Always writable
  - Conflict are resolved at read time
- Who resolve the conflict
  - The data store; limited
  - The application; more complex

# Other Key design principles

- *Incremental scalability* - be able to scale out one storage host at a time
- *Symmetry* - every node in Dynamo should have the same set of responsibilities as its peers
- *Decentralization* - favor decentralized peer-to-peer techniques over centralized control
- *Heterogeneity* - to exploit heterogeneity in the infrastructure

# System Architecture

- System interface
- Partitioning Algorithm
- Replication
- Data Versioning

# System Interface

- Get(key)
  - locates the object replicas associated with the key in the storage system
  - returns a single object or a list of objects with conflicting versions along with a context

- Put(key, context, object)
  - determines where the replicas of the object should be placed based on the associated key
  - writes the replicas to disk

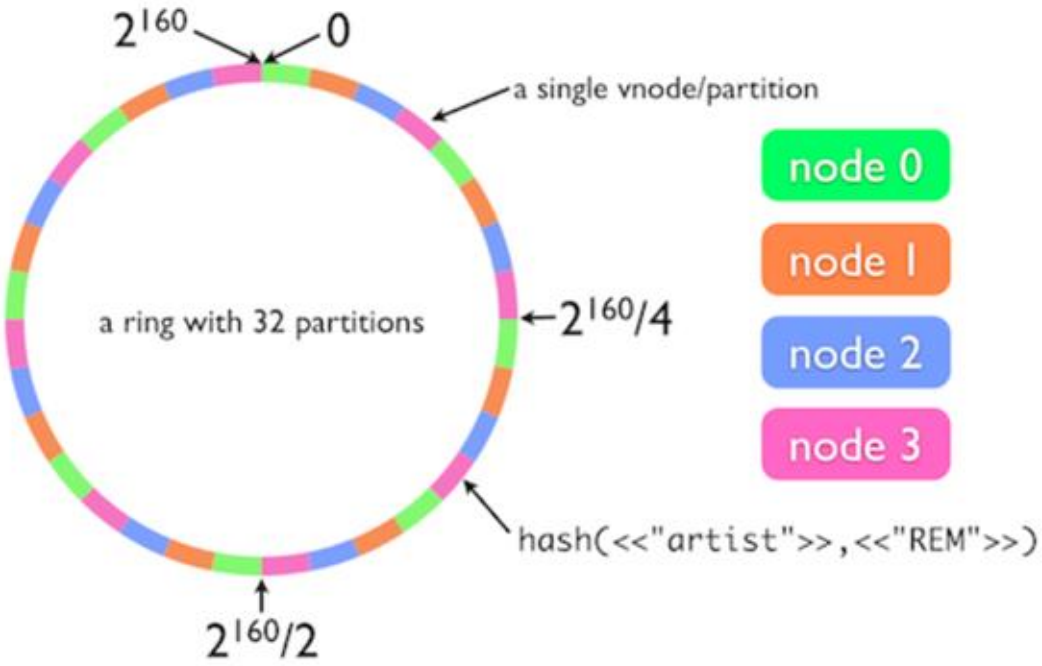
- The context encodes system metadata about the object that is opaque to the caller and includes information such as the version of the object

# Partitioning Algorithm

- To scale incrementally
  - Dynamic partitioning of data over a set of nodes
- Partitioning with consistent hashing to distribute the load across multiple storage hosts
  - Each node in the system is assigned a random value within this space which represents its “position” on the ring. Each data item identified by a key is assigned to a node by hashing the data item’s key to yield its position on the ring
- Challenge
  - Non uniform distribution and unbalance of the load

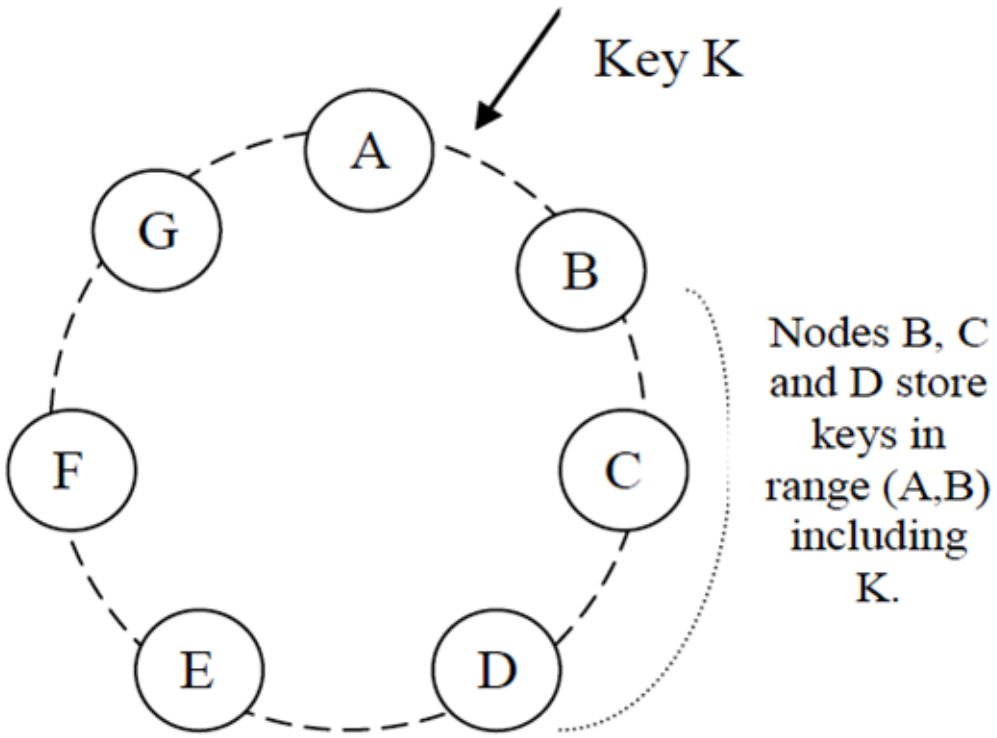
# Partitioning algorithm (cont'd)

- The concept of virtual node is introduced
- If a node becomes unavailable
  - the load handled by this node is evenly dispersed across the remaining available nodes
- When a node becomes available again, or a new node is added to the system
  - the newly available node accepts a roughly equivalent amount of load from each of the other available nodes
- The number of virtual nodes that a node is responsible can decided based on its capacity, accounting for heterogeneity in the physical infrastructure



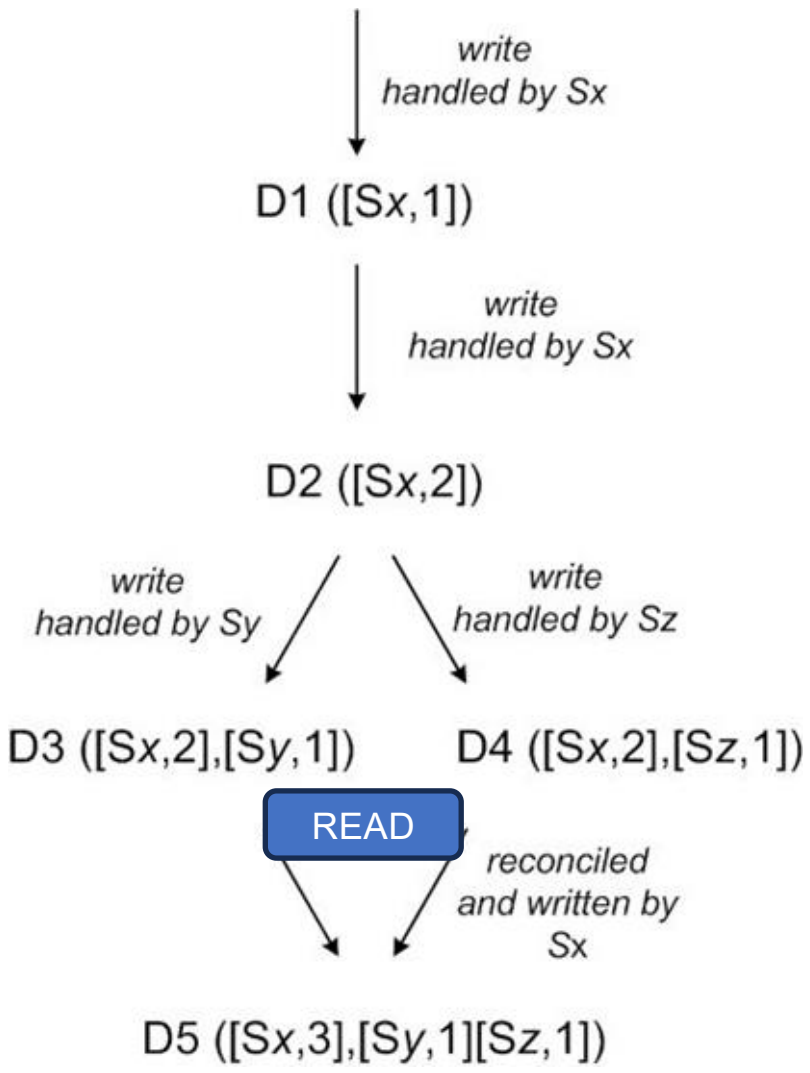
# Replication

- To achieve high availability and durability data is replicated on multiple hosts
  - Each data item is replicated at N hosts
- Each key, k, is assigned to a coordinator node
  - in charge of the replication of the data items that fall within its range
  - In addition to locally storing each key within its range, the coordinator replicates these keys at the N-1 clockwise successor nodes in the ring



# Data Versioning

- Eventual consistency allows for updates to be propagated to all replicas asynchronously
- A put() call can return before all the replicas are updated
  - A subsequent get() may return an object that does not have the latest update
- Some application can tolerate such behavior, e.g. “add to chart”
  - An add can never be rejected
  - A change can be done on an older/not-updated version of the chart
  - Divergent versions will be reconciled
- The result of each modification is treated as a new and immutable version
  - Vector clock (node, counter) are used to mark the version and capture causality



# Questions?

Feedback?